

UNIVERSIDAD POLITÉCNICA DE MADRID

TRABAJO FINAL DE MINT

Análisis de la Red de Dependencias de Python

Bogurad Barański Barańska
Jorge Bengoa Pinedo

5 de mayo de 2026

Índice general

1. Introducción	1
1.1. Descripción de la Red Propuesta	1
1.2. Construcción de la Red	1
1.3. Herramientas Utilizadas	1
1.3.1. Características del Equipo	1
1.3.2. Software empleado	2
1.4. Objetivos del Estudio	2
2. Análisis de la red	3
2.1. Descripción de la red	3
2.1.1. Distribución de grado	9
2.2. Análisis de centralidad	15
2.2.1. PageRank	16
2.2.2. Centralidad de Intermediación	18
2.2.3. Centralidad de Cercanía	19
2.2.4. Distribuciones de Centralidad	21
2.3. Análisis de comunidades	22
3. Herramienta de visualización de la red	29
3.1. Arquitectura general	29
3.2. Cálculos preliminares	29
3.3. Motor de físicas	30
3.4. Métricas y modos de color	32
3.5. Modo de visualización de ciclos	33
3.6. Inversión de colores	33
3.7. Interacción	33
3.7.1. Controles de teclado	33
3.7.2. Inspección de nodos	34
4. Conclusiones	35

Capítulo 1

Introducción

1.1. Descripción de la Red Propuesta

En este trabajo se va a analizar la red de dependencias del ecosistema de paquetes de Python. La red consiste en un grafo dirigido donde los nodos representan paquetes de software y las aristas dirigidas representan las dependencias entre ellos (un paquete requiere de otro para su instalación o funcionamiento).

1.2. Construcción de la Red

La red se construye mediante un proceso de extracción de datos automatizado (*web scraping*) haciendo uso de la API pública en formato JSON de PyPI mediante el script `create_network.py`. Se parte de una lista de 10 paquetes semilla representativos de distintas áreas, y se aplica un algoritmo de búsqueda recursivo. Por cada paquete, se extraen sus dependencias directas de forma asíncrona, añadiéndolas a la cola de procesamiento hasta alcanzar el volumen deseado para el estudio.

1.3. Herramientas Utilizadas

En esta sección se detallan las características del Hardware empleado junto con las versiones del Software utilizado.

1.3.1. Características del Equipo

Se ha utilizado un portátil con las siguientes características:

- **Sistema Operativo:** Microsoft Windows 11 Home (Versión 10.0.26200)
- **Procesador (CPU):** Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz
- **Memoria RAM:** 16 GB

1.3.2. Software empleado

Como lenguaje de programación se ha utilizado **Python** en la versión **3.13.7**. Las librerías y dependencias principales utilizadas son:

- `networkx==3.6.1`: Para la construcción de la red y procesado preliminar.
- `igraph==1.0.0`: Para el cálculo eficiente en grafos.
- `pandas==2.3.2`: Para la estructuración y manejo de los datos.
- `aiohttp==3.13.3`: Para la ejecución segura de peticiones web asíncronas a la API de PyPI durante la fase de recolección de datos.
- `vispy==0.16.1`: Para el renderizado interactivo del motor de físicas del grafo, haciendo uso de la aceleración gráfica (GPU).
- `numpy==2.3.3`: Para el manejo eficiente de operaciones matriciales y vectorización de las fuerzas en el motor de renderizado.
- `matplotlib==3.10.7`: Para la generación visual de tablas y gráficos estáticos de soporte durante el análisis.

1.4. Objetivos del Estudio

Los objetivos principales de este análisis son:

1. Obtener una visión macroscópica de la estructura del ecosistema de desarrollo de Python.
2. Identificar los paquetes *hub* o nodos centrales que actúan como dependencias críticas transversales para la mayoría de los proyectos.
3. Detectar comunidades o clústeres de paquetes que tienden a utilizarse conjuntamente dentro de dominios de desarrollo específicos.

Capítulo 2

Análisis de la red

2.1. Descripción de la red

En la red de dependencias de Python estudiada los nodos representan las librerías mientras que las aristas son las dependencias. De esta forma, un nodo A apuntará a otro nodo B si depende de él $A \rightarrow B$. Si B , además, depende de C pero A lo hace a través de B , no existe el enlace entre A y C . Las figuras 2.1 y 2.2 ilustran la red completa y un zoom de su centro, respectivamente.

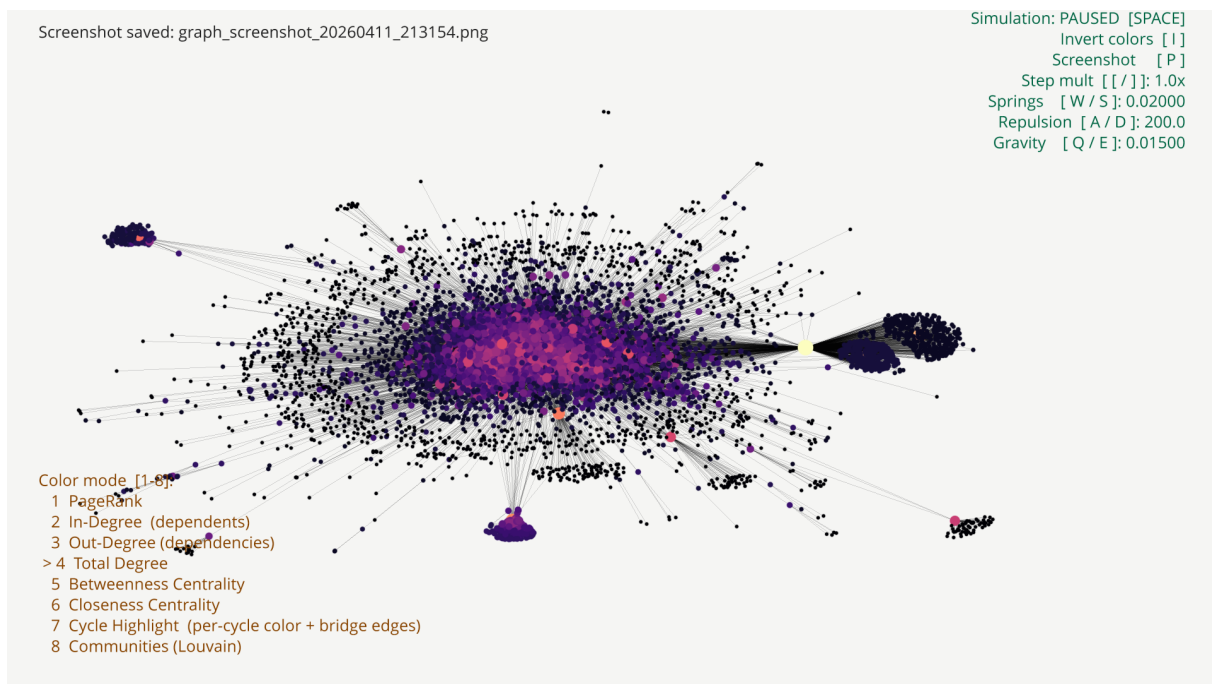


Figura 2.1: Representación del grado total de los nodos de la red explotada en su totalidad.

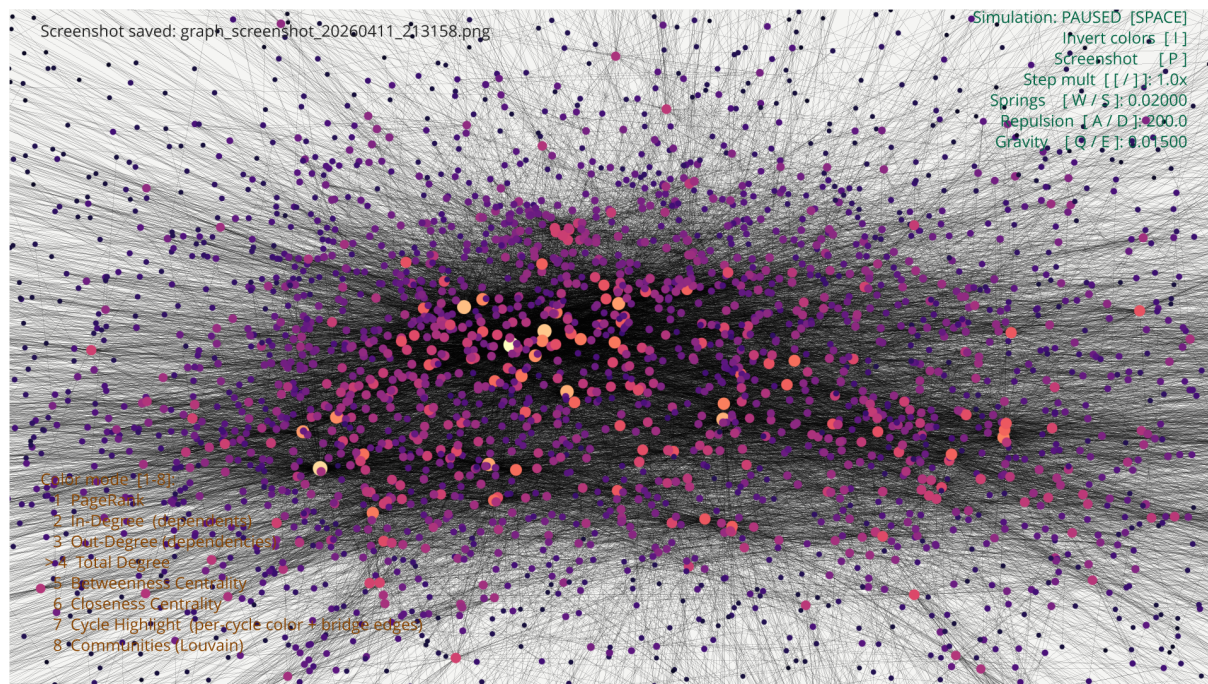


Figura 2.2: Representación del grado total de los nodos de la red explotada con zoom a la agrupación central.

La red analizada es dirigida con 6765 nodos y 36054 aristas. Visualmente se representa el grado de entrada de los nodos (figuras 2.3 y 2.4) y de salida (figuras 2.5 y 2.6). La densidad es muy baja, 0,000788, lo que encaja con la idea de que los desarrolladores incluyen un número pequeño de dependencias en sus programas respecto del total de la red.

El coeficiente de clustering medio es de 0.3170. Dado que es un valor muy superior a la densidad de la red se puede decir que el clustering es alto. La propiedad de mundo pequeño se ve perjudicada, pues las dependencias de un paquete tienen mayor probabilidad de depender entre sí. Posiblemente, las distancias entre distintos puntos de la red tiendan a ser grandes. Además, el coeficiente global de clustering es 0.03584. Esto sugiere que los paquetes forman grupos aislados que se conectan a través de hubs con muchas conexiones. Sin embargo, los nodos de un grupo no están conectados directamente a los de otro grupo.

El comportamiento descrito tiene sentido desde la perspectiva del desarrollo de paquetes. Los hubs son librerías conocidas que dan acceso a *frameworks* enteros. Los desarrolladores incluyen los hubs en sus programas, no cada una de las dependencias menos conocidas. Los paquetes dentro de un mismo *framework* tienen un objetivo en común, por lo que interactúan entre sí para conseguirlo. Adicionalmente, podrán interactuar con hubs a otros *frameworks*, pero no directamente con cada una de las librerías que contienen.

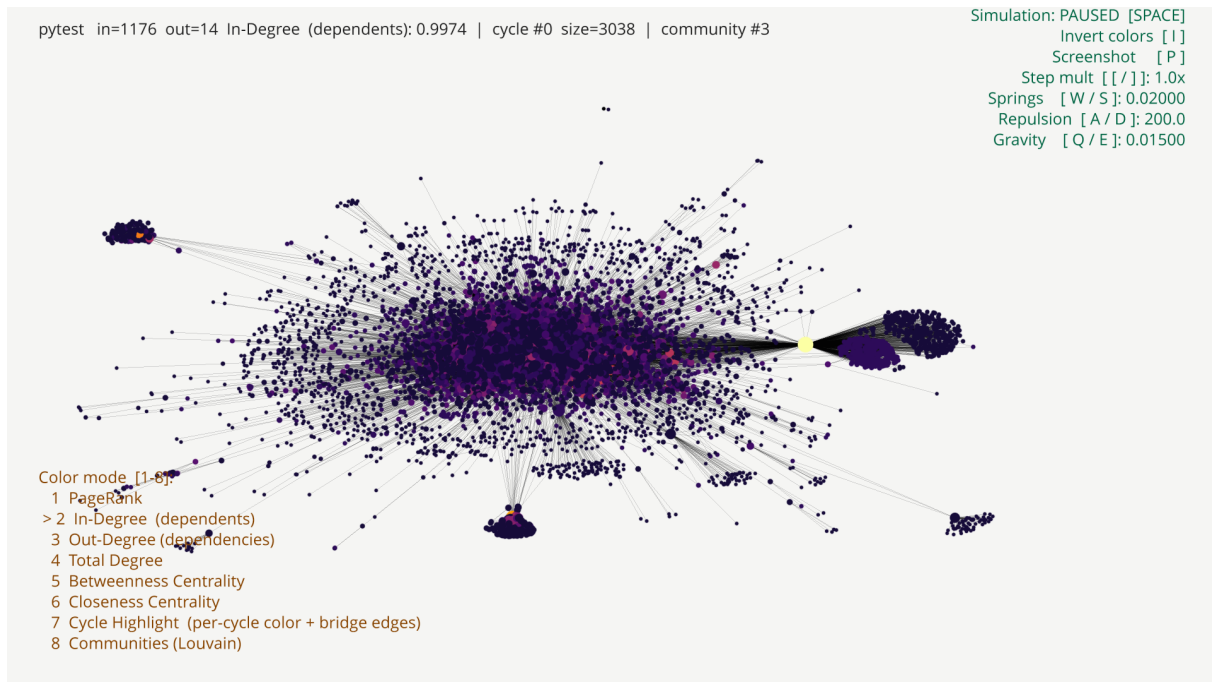


Figura 2.3: Representación del grado de entrada de los nodos de la red explotada en su totalidad.

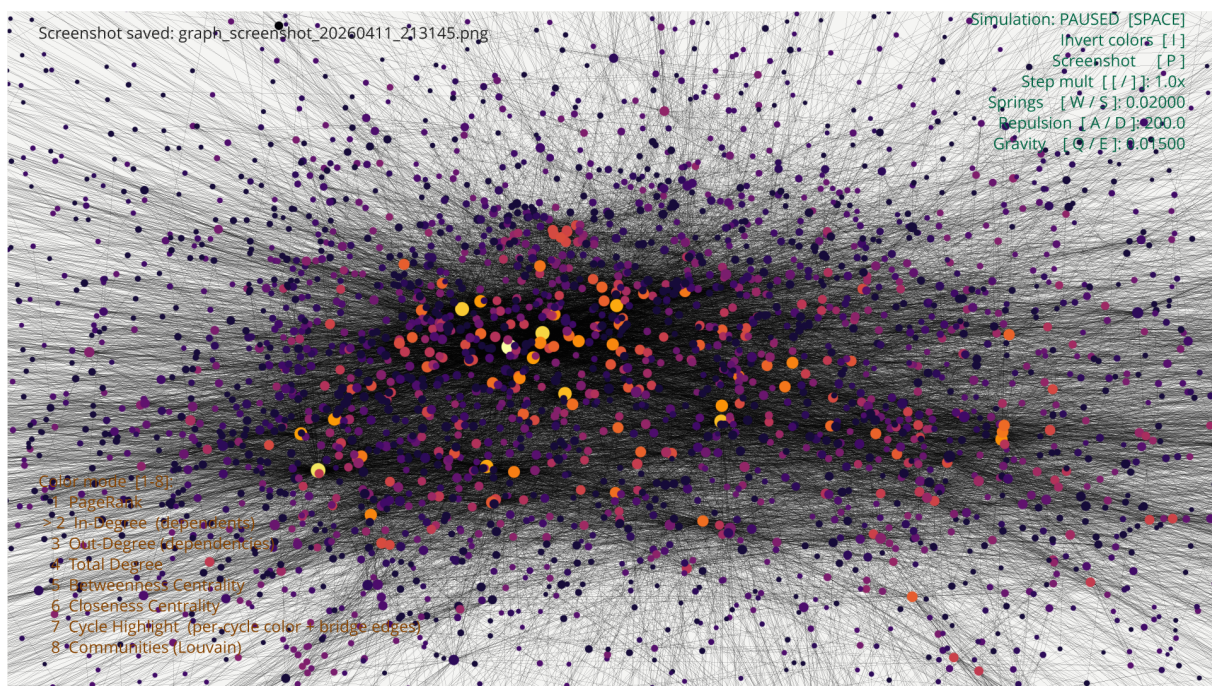


Figura 2.4: Representación del grado de entrada de los nodos de la red explotada con zoom a la agrupación central.

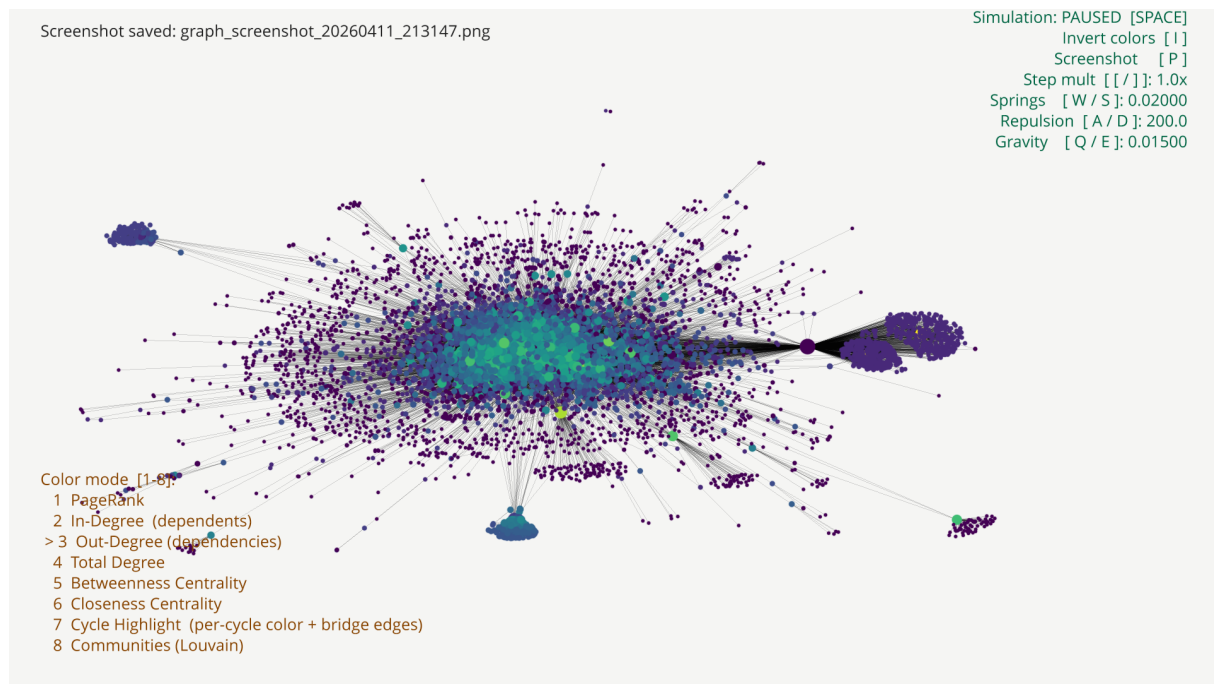


Figura 2.5: Representación del grado de salida de los nodos de la red explotada en su totalidad.

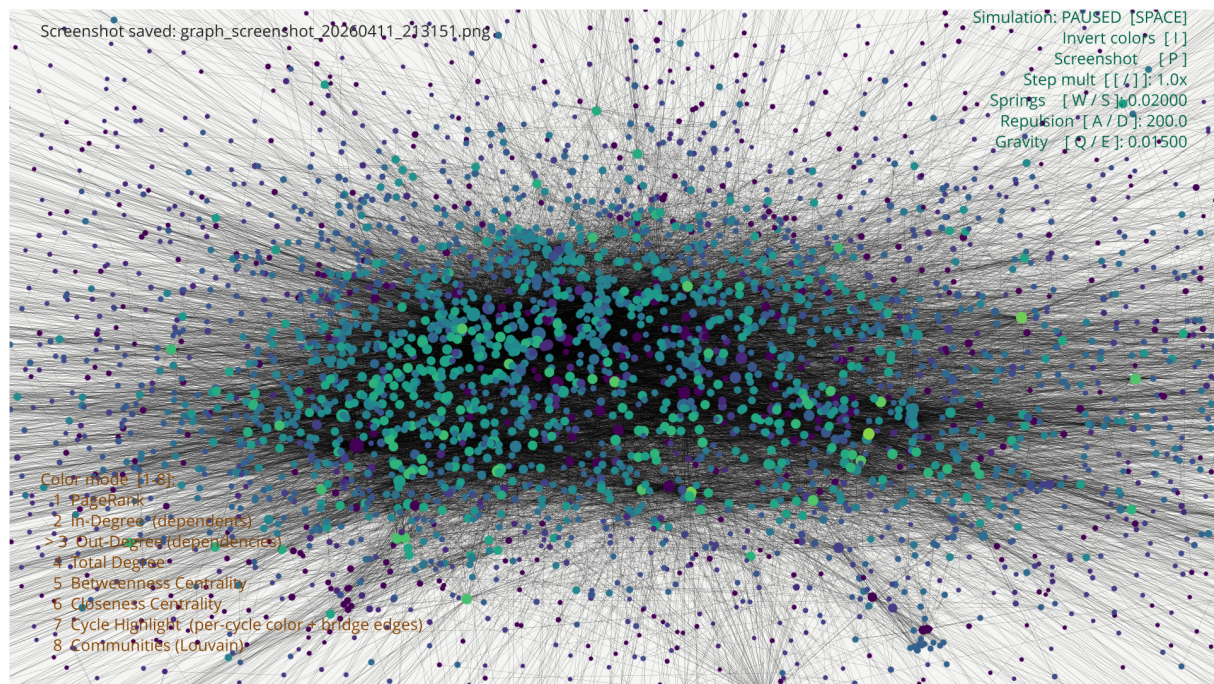


Figura 2.6: Representación del grado de salida de los nodos de la red explotada con zoom a la agrupación central.

De entre todos los nodos, un total de 6763(99,97%) pertenecen a la componente gigante. Los paquetes `ansible` y `ansible-core~` pertenecen a una isla desconectada del resto del grafo. Resulta extraño que `ansible`, un paquete de automatización ampliamente

utilizado en la industria para configurar servidores y servicios por lotes, no tenga dependencias de otros paquetes. La explicación se encuentra en la forma de construir la red. La dependencia de `ansible` es `ansible-core`, que a su vez depende de otros paquetes. No obstante, la expresión regular del parser no filtra adecuadamente el `~` y no encuentra el paquete `ansible-core~` porque no existe. Las figuras 2.7 y 2.8 representan la componente gigante.

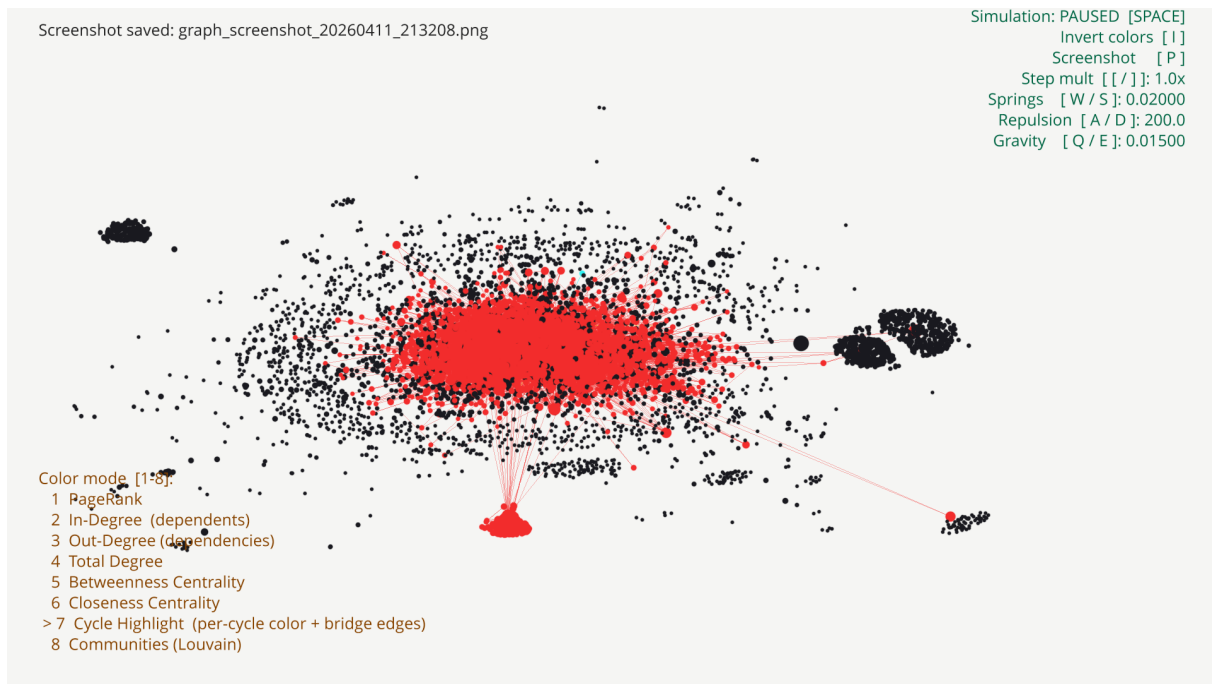


Figura 2.7: Representación de la componente fuertemente conexa en rojo de la red explotada.

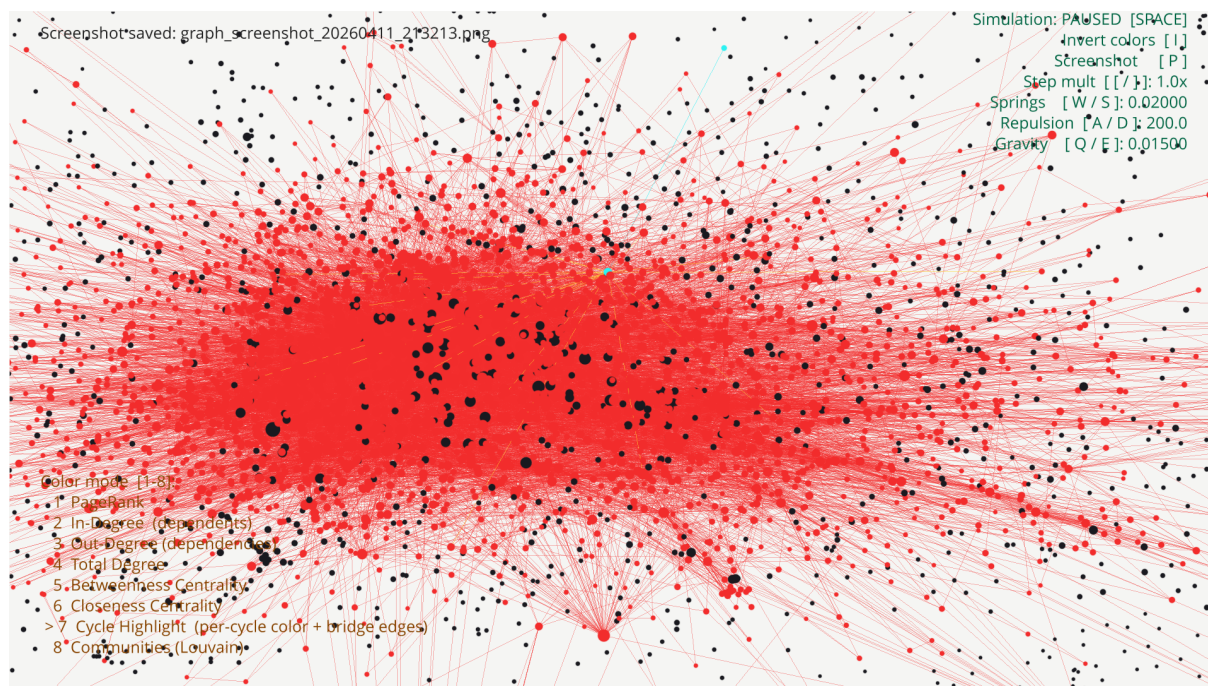


Figura 2.8: Representación de la componente fuertemente conexa en rojo con zoom en la agrupación central.

La detección de islas se realiza analizando las componentes débilmente conexas, que no tienen en cuenta la dirección. Otro análisis interesante es el de las componentes fuertemente conexas. Se dice que la conexión entre dos nodos es fuerte si se puede ir y volver entre ellos. Este concepto está relacionado con la existencia de bucles. Si hay paquetes que se necesitan entre sí no se tiene un punto claro de inicio, por lo que la instalación no es limpia. Además, una actualización en uno de los paquetes podría provocar un cambio en el otro, y este cambio a su vez otro cambio en el paquete original. Este ciclo podría llegar a repetirse infinitamente.

La cantidad de componentes fuertemente conexas debería ser igual al número de nodos (no hay paquetes que dependan mutuamente, por lo que no pueden formar una componente). En la red analizada hay 3727, lo que sugiere la existencia de ciclos. Solamente 2 componentes tienen más de un nodo. Es decir, hay 3725 paquetes sin dependencias cíclicas y los 3040 restantes se reparten entre 2 bucles. La longitud media de los bucles es de 1520 paquetes. Afinando el análisis, uno de los bucles contiene 2 nodos y el otro por 3038. El primero está compuesto por `ryd` y `ruamel.yaml`, dos paquetes de DevOps empleados para realizar lecturas y escrituras sobre archivos YAML. La composición del segundo es mucho más amplia como para analizar paquete a paquete. En la figura 2.9 se han clasificado las librerías del bucle según la categoría a la que pertenecen utilizando un modelo de lenguaje. Cabe decir que dentro de la propia componente fuertemente conexa podría haber bucles internos, aunque este análisis no se ha realizado. Observando alguno de los paquetes en la componente fuertemente conexa, algunos de ellos resultan conocidos (por ejemplo, `fastapi`, `django` o `requests`) e incluso por experiencia al instalarlos no han arrastrado al resto de elementos en el bucle. La razón de este comportamiento es de nuevo atribuible a la construcción de la red, que no discrimina entre dependencias de

producción o desarrollo. En producción, las instalaciones siguen una jerarquía de dependencias donde solamente se descargan los paquetes necesarios para el funcionamiento. En desarrollo, los autores de los paquetes base suelen declarar dependencias hacia paquetes de más alto nivel para comprobar que su código se integra correctamente con ellos.

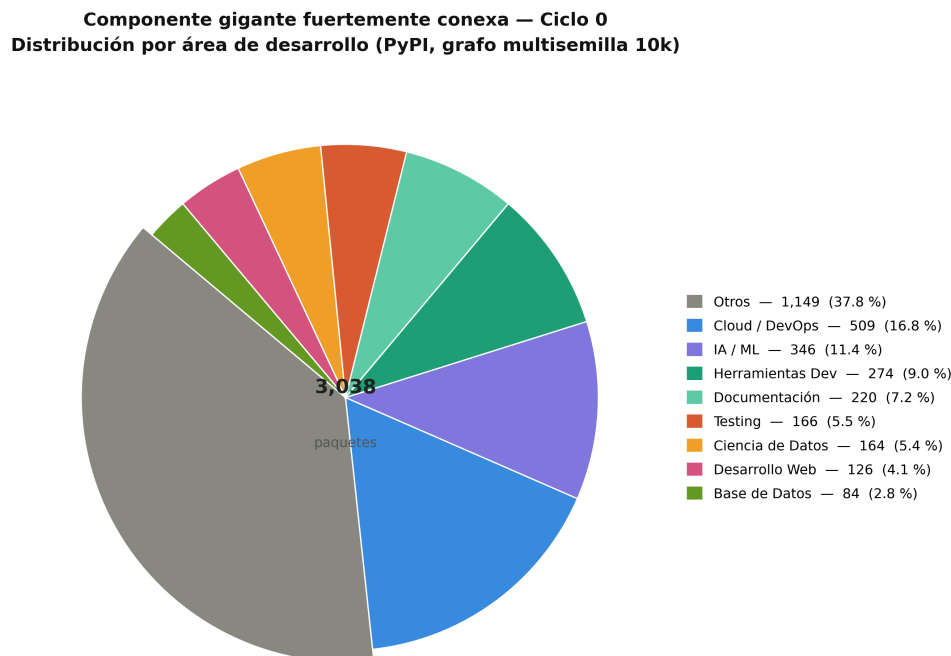


Figura 2.9: Categorización de los paquetes de PyPI en la componente fuertemente conexas.

En la componente gigante extraída anteriormente se analizan algunas métricas adicionales. El diámetro es 25, lo que implica que en el peor de los casos esa será la profundidad de las dependencias de un paquete¹. El radio se distingue según sea de entrada o de salida. El valor del primero es 6 y se relaciona con la accesibilidad de los paquetes básicos. Cualquier paquete dependiente del paquete menos excéntrico, lo alcanzará en un máximo de 6 pasos. El valor del segundo es 13 y se relaciona con la facilidad de instalar paquetes. En el mejor de los casos (paquete mejor posicionado), se requiere una instalación de profundidad 13.

2.1.1. Distribución de grado

Este apartado se ha realizado conforme al artículo [1].

¹No se debe asociar el diámetro al número de dependencias, sino a la profundidad de las mismas. Por ejemplo, un paquete podría depender de varios paquetes que no dependen de nadie más, en cuyo caso la profundidad sería 1 pero el número de dependencias sería mayor que 1.

Estimación de parámetros

El objetivo es verificar si la distribución de grado sigue una ley de potencias. Dado un valor k_{mn} a partir del cual se empieza a cumplir la ley de potencias, se estima el valor de α por máxima verosimilitud resolviendo la ecuación:

$$\frac{\zeta'(\hat{\alpha}, x_{\min})}{\zeta(\hat{\alpha}, x_{\min})} = -\frac{1}{n} \sum_{i=1}^n \ln x_i \quad (2.1)$$

donde $\zeta(\alpha, x_{mn})$ es la función zeta generalizada de Hurwitz. En vez de resolver la ecuación anterior de forma numérica se puede obtener una aproximación a partir de la siguiente expresión:

$$\hat{\alpha} \approx 1 + n \left[\sum_{i=1}^n \ln \frac{x_i}{x_{mn} - 0,5} \right] \quad (2.2)$$

donde n es el número de paquetes en la cola y x_i es el grado de la i -ésima observación. La desviación típica de $\hat{\alpha}$ en tiempo discreto se calcula con la fórmula:

$$\sigma = \frac{1}{\sqrt{n \left[\frac{\zeta''(\hat{\alpha}, x_{\min})}{\zeta(\hat{\alpha}, x_{\min})} - \left(\frac{\zeta'(\hat{\alpha}, x_{\min})}{\zeta(\hat{\alpha}, x_{\min})} \right)^2 \right]}} \quad (2.3)$$

No obstante, si el número de nodos en la cola es alto se puede aproximar por:

$$\hat{\sigma} \approx \frac{\hat{\alpha} - 1}{\sqrt{n}} \quad (2.4)$$

La elección del valor de x_{mn} se hace por *grid search*. Es decir, se obtiene el estimador de máxima verosimilitud de α para cada valor posible de x_{mn} y se selecciona el que minimice el estadístico de Kolmogorov-Smirnov:

$$KS = \max_{x \geq x_{mn}} |S(x) - P(x)| \quad (2.5)$$

donde $S(x)$ es la función de distribución empírica de la cola de los datos disponibles y $P(x)$ es la función de distribución teórica de la ley de potencias. Si el valor de x_{mn} es alto se tendrán pocos valores en la cola y el ajuste será deficiente debido al ruido estadístico. Si es bajo, se estará modelando el parte del cuerpo y la cola con una distribución que describe solo la cola. Entre medias se encuentra el valor óptimo que se pretende estimar, $\hat{x}_{mn}$.

La incertidumbre asociada a $\hat{x}_{mn}$ se cuantifica empleando *bootstrapping* no paramétrico. Se construyen conjuntos de datos sintéticos mediante muestreo con reemplazo sobre el conjunto original y se busca $\hat{x}_{mn}$ en cada caso con el método explicado anteriormente. La incertidumbre asociada a $\hat{x}_{mn}$ es la desviación típica sobre los conjuntos sintéticos.

Tests de bondad de ajuste

Los métodos anteriores no explican la plausibilidad de la ley de potencias calculada. El problema es complejo por dos razones. La primera es que incluso las propias realizaciones a partir de una ley de potencias dadas podrían no seguir dicha ley debido al

proceso de muestreo. La segunda es que se pueden dar casos en los que la red proviene de otra distribución pero su realización particular sigue una ley de potencias. Se busca plantear un test de bondad de ajuste que permita discriminar entre fluctuaciones aleatorias asociadas al muestreo y distribuciones distintas. El procedimiento se basa en generar varios conjuntos de datos sintéticos y calcular la proporción de veces que el estadístico de Kolmogorov-Smirnov es menor en los datos empíricos que en los generados artificialmente. Este cociente es el p-valor.

El contraste anterior es semiparamétrico, dado que con probabilidad $\frac{n_{cola}}{n}$, con n_{cola} el número de observaciones en la cola dado el punto de corte $\hat{x}_{mn}$, se generan datos de acuerdo a la ley de potencias y el resto se seleccionan aleatoriamente del cuerpo de los datos originales. Para cada conjunto sintético se reajustan los valores de $\hat{x}_{mn}$ y $\hat{\alpha}$ con el fin de seguir el mismo procedimiento que se siguió con la red original². Para muestrear de una ley de potencias discreta, habría que aplicar el método de la transformada inversa sobre:

$$P(x) = \sum_{x'=x}^{\infty} p(x') = 1 - r \quad (2.6)$$

donde $r \sim U(0,1)$. La fórmula anterior no es invertible, por lo que debe resolverse numéricamente. Una alternativa es utilizar una aproximación:

$$x = \left\lfloor \left(x_{\min} - \frac{1}{2} \right) (1 - r)^{-1/(\alpha-1)} + \frac{1}{2} \right\rfloor \quad (2.7)$$

Comparación con distribuciones alternativas

Incluso si la ley de potencias ajusta adecuadamente la distribución de grado, podría haber distribuciones que lo hicieran mejor. La forma de proceder es obtener la distribución alternativa que mejor se ajusta a los datos y comparar el cociente de log-verosimilitudes, $\mathcal{R}$. Si $\mathcal{R}$ es positivo, la ley de potencias se ajustará mejor a los datos, mientras que si es negativo será la distribución alternativa. Si es cercano a 0 significa que ha habido un empate. El signo de $\mathcal{R}$ podría deberse meramente al ruido estadístico, por lo que se debe calcular su valor esperado. Otro aspecto a tener en cuenta consiste en determinar qué significa que $\mathcal{R}$ esté cerca o lejos de 0. La respuesta viene del test de Voung.

A continuación, se expone el formalismo matemático:

²Si la red original proviniera de una ley de potencias, pero los datos en el cuerpo se alinearan con los de la cola, el estadístico KS sería más bajo de lo que debería. Por lo tanto, el p-valor estaría inflado debido a que el sobreajuste provoca un error menor que el que tendrían los datos sintéticos al medirse sobre la ley que los generó. Para que la comparación sea justa, los parámetros de la ley de potencias deben reajustarse.

$$\left. \begin{aligned} L_1 &= \prod_{i=1}^n p_1(x_i) \\ L_2 &= \prod_{i=1}^n p_2(x_i) \\ R &= \frac{L_1}{L_2} = \prod_{i=1}^n \frac{p_1(x_i)}{p_2(x_i)} \end{aligned} \right\} \Rightarrow \mathcal{R} = \sum_{i=1}^n [\ln p_1(x_i) - \ln p_2(x_i)] = \sum_{i=1}^n [\ell_i^{(1)} - \ell_i^{(2)}] \quad (2.8)$$

Dado que las muestras x_i son independientes, las diferencias $\ell_i^{(1)} - \ell_i^{(2)}$ también lo son. Aplicando el teorema central del límite $\mathcal{R}$ se distribuye como una normal con varianza $n\sigma^2$, donde σ^2 es la varianza de un término que se puede aproximar por la varianza muestral:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n \left[\left(\ell_i^{(1)} - \ell_i^{(2)} \right) - \left(\hat{\ell}^{(1)} - \hat{\ell}^{(2)} \right) \right] \quad (2.9)$$

$$\hat{\ell}^{(1)} = \frac{1}{n} \sum_{i=1}^n \ell_i^{(1)} \quad (2.10)$$

$$\hat{\ell}^{(2)} = \frac{1}{n} \sum_{i=1}^n \ell_i^{(2)} \quad (2.11)$$

Si las distribuciones son iguales, la diferencia de log-verosimilitudes es 0, que se toma como hipótesis nula. Esta se rechazará si la probabilidad de obtener la diferencia de log-verosimilitudes observada es menor que un determinado valor:

$$p = \frac{1}{\sqrt{2\pi n\sigma^2}} \left[\int_{-\infty}^{-|R|} e^{-\frac{t^2}{2n\sigma^2}} dt + \int_{|R|}^{\infty} e^{-\frac{t^2}{2n\sigma^2}} dt \right] \quad (2.12)$$

Resultados

La tabla 2.1 resume el ajuste de los parámetros de la ley de potencias y la simulación de Monte Carlo para evaluar la bondad del ajuste en la distribución de los grados de entrada y salida de la red. La hipótesis de distribución bajo una ley de potencias se verifica para el grado de entrada pero se rechaza para el de salida. Esto es coherente con la idea de que si alguien desarrolla un nuevo paquete, este dependa de los paquetes populares antes que de los desconocidos. Sin embargo, un paquete con muchas dependencias no va a depender de nuevos paquetes que se incorporen a la red (principalmente porque ya podía funcionar sin dichos paquetes).

El valor de $\hat{x}_{mn}$ también puede ser indicador del cumplimiento de la ley de potencias. Para el grado de entrada se mantiene en el rango $[2, 4]$ mientras que para el de salida las fluctuaciones son mayores. Además, el estadístico KS es 6 más alto en la segunda distribución. Cabe decir que $\alpha < 2$ para la distribución de grado de entrada, lo que implica que todos los momentos divergen, incluidos la media y la varianza. Por lo tanto, no se puede hablar de grado medio.

Métrica	Grado de Entrada	Grado de Salida
$x_{\min}$ óptimo	3.0	11.0
$x_{\min}$ bootstrap	$3,6 \pm 1,4$	$21,4 \pm 8,9$
α óptimo	$1,8999 \pm 0,0222$	$2,5158 \pm 0,0479$
KS óptima	0.0139	0.0608
n_{cola}	1646 (24.3 %)	1001 (22.0 %)
p-valor (Test KS)	0.532	0.000
Test de Vuong	No rechazada ($p > 0,1$)	Rechazada ($p \leq 0,1$)

Tabla 2.1: Resumen del ajuste de ley de potencias para los grados de entrada y salida.

La figura 2.10 muestra las distribuciones de masa de probabilidad empírica y teórica de los grados de entrada y salida de la red. Visualmente, el ajuste parece mejor para el grado de entrada. Hay que enfatizar que la distribución teórica se calcula solo para los datos de la cola, los cuales representa una fracción de la probabilidad total. Si se dibuja la ley de potencias calculada directamente saldrá por encima de la nube de puntos. Esto es porque la línea negra acumulará toda la probabilidad y tendrá más para repartir a cada punto. Para corregir el efecto, se debe multiplicar la probabilidad de grado teórica por la probabilidad de pertenecer a la cola ($\frac{n_{\text{cola}}}{n}$). También se ve cómo a partir de un determinado grado la probabilidad de la distribución empírica se estanca, mientras que la de la teórica sigue decreciendo. La razón es el número limitado de observaciones disponibles. Los paquetes con un elevado número de dependencias aparecen una vez en la red, sin importar cuál sea ese número. Por lo tanto, la probabilidad que acumulan es la misma. Si hubiera más muestras se seguiría la ley de potencias más allá de lo que se ve en la imagen. Finalmente, a la derecha se representa el valor del estadístico KS según el valor de corte $\hat{x}_{mn}$.

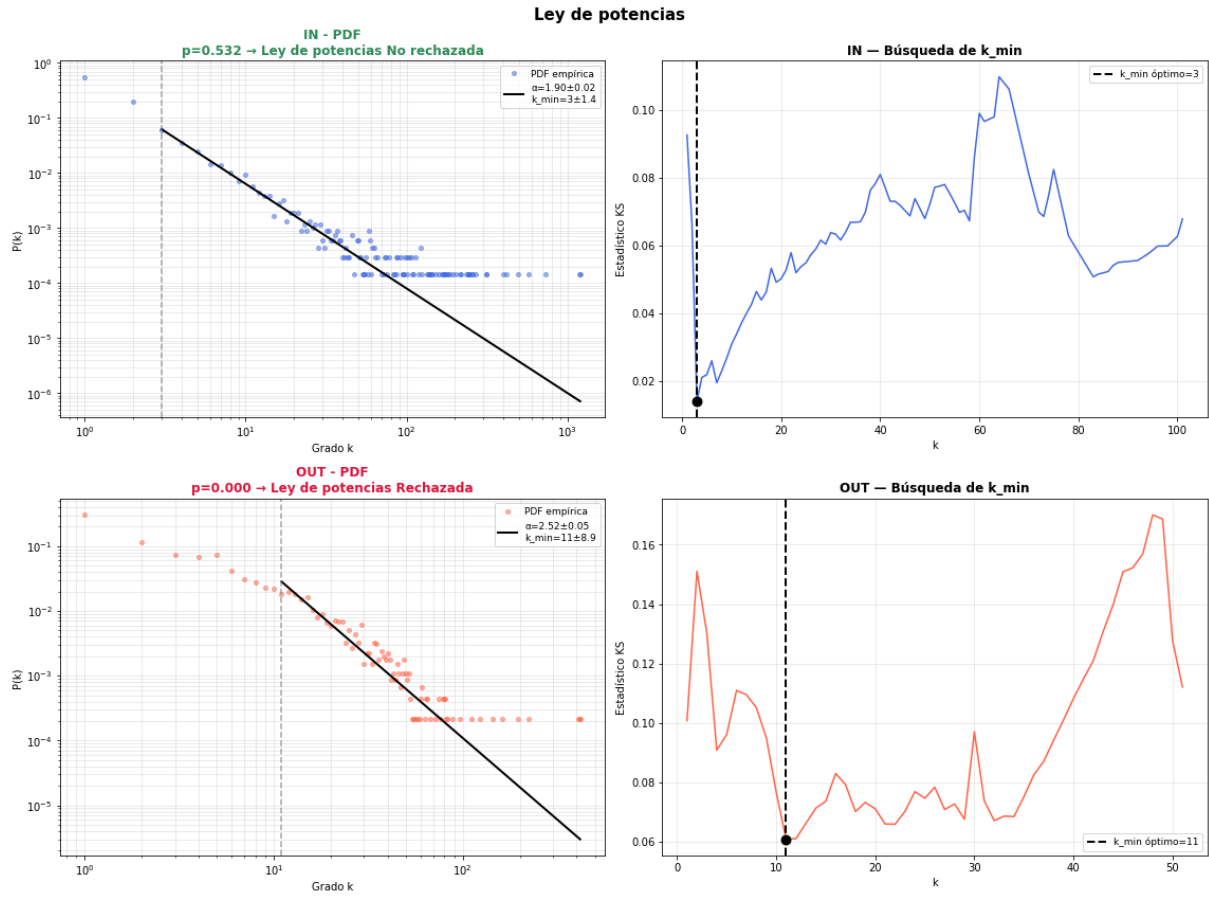


Figura 2.10: Distribuciones de masa de probabilidad empíricas y teóricas y proceso de búsqueda de x_{mn} óptimo para los grados de entrada y salida.

La distribución según ley de potencias se compara con las presentadas en la tabla 2.2. En la tabla 2.3 se resumen los tests de razón de verosimilitud entre la ley de potencias y las distribuciones alternativas. Aunque para la distribución de grado de salida no tiene demasiado sentido porque se rechazó la hipótesis de que siguen una ley de potencias, la comparación sugiere que una Yule podría funcionar mejor. Para la distribución de entrada, la ley de potencias resulta favorable frente a la exponencial y la Poisson, mientras que es indistinguible de la Yule.

Distribución	Ecuaciones (PMF y log-verosimilitud)
Exponencial	$p(x) = (1 - e^{-\lambda})e^{\lambda x_{\min}}e^{-\lambda x}$ $\ell(x) = \ln(1 - e^{-\lambda}) + \lambda x_{\min} - \lambda x$
Yule	$p(x) = (\alpha - 1) \frac{\Gamma(x_{\min} + \alpha - 1)}{\Gamma(x_{\min})} \frac{\Gamma(x)}{\Gamma(x + \alpha)}$ $\ell(x) = \ln(\alpha - 1) + \ln \Gamma(x_{\min} + \alpha - 1) - \ln \Gamma(x_{\min}) + \ln \Gamma(x) - \ln \Gamma(x + \alpha)$
Poisson	$p(x) = \left[e^{\mu} - \sum_{k=0}^{x_{\min}-1} \frac{\mu^k}{k!} \right]^{-1} \frac{\mu^x}{x!}$ $\ell(x) = x \ln(\mu) - \ln \Gamma(x + 1) - \ln \left(e^{\mu} - \sum_{k=0}^{x_{\min}-1} \frac{\mu^k}{k!} \right)$

Tabla 2.2: Funciones de masa de probabilidad y log-verosimilitud para distribuciones discretas alternativas a la ley de potencias: exponencial, Yule y Poisson.

Distribución Alternativa	LR ($\mathcal{R}$)	p-valor	Interpretación
Grado de Entrada	$(k_{\min} = 3, \alpha = 1,900, p_{\text{KS}} = 0,532)$		
Exponencial	8,432	< 0,001	Ley de potencias favorable
Poisson	5,401	< 0,001	Ley de potencias favorable
Yule	-1,598	0,110	No distinguibles
Grado de Salida	$(k_{\min} = 11, \alpha = 2,516, p_{\text{KS}} < 0,001)$		
Exponencial	1,307	0,191	No distinguibles
Poisson	3,824	< 0,001	Ley de potencias favorable
Yule	-8,943	< 0,001	Alternativa favorable

Tabla 2.3: Tests de razón de verosimilitud entre la ley de potencias empírica y las distribuciones alternativas para la región de la cola.

2.2. Análisis de centralidad

Una vez realizado el análisis preliminar de la red se estudian las propiedades de centralidad de la misma para ver qué paquetes son los más importantes dentro de la red de dependencias de Python. Para realizar este estudio se estudian las tres magnitudes estudiadas en clase: PageRank, Centralidad de Intermediación y Centralidad de Cercanía mediante el script `centrality.py`.

2.2.1. PageRank

Como se trabaja con un grafo dirigido, esta medida de centralidad es de aplicación directa al análisis de trabajo. En este contexto, el PageRank representa la probabilidad de alcanzar un paquete dado mediante un paseo aleatorio de Markov a través del grafo, utilizando un factor de salto de $\theta = 0,85$. Por tanto, puede entenderse que aquellos paquetes que acaben con un valor más alto de esta métrica serán los más relevantes de la red de dependencias de Python.

Visualmente, tal y como se puede ver en las figuras 2.11 y 2.12 se ven varios nodos que destacan en importancia. Además, la ejecución del script `centrality.py` confirma que hay dos nodos, `typing-extensions` y `pytest`, que presentan una ponderación logarítmica muy superior al resto. Estos son seguidos por paquetes donde ya la distancia en PageRank entre ellos ya no es tan grande.

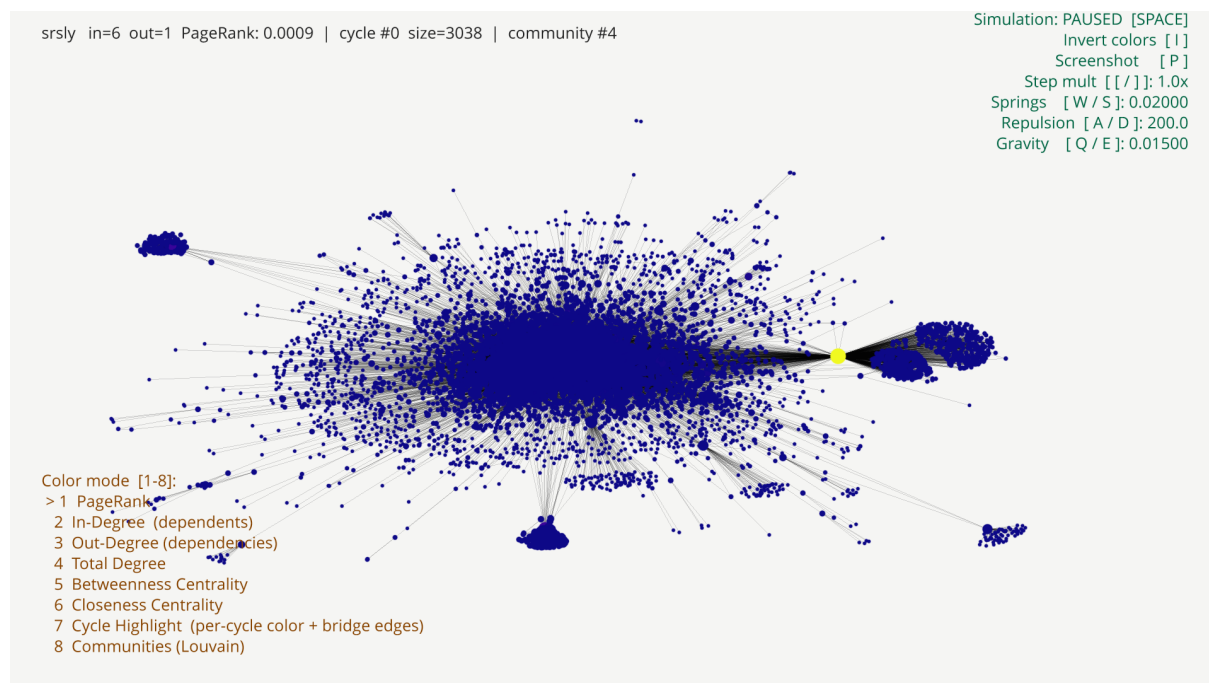


Figura 2.11: Representación del PageRank en la red completa de dependencias de Python donde en amarillo están los paquetes con mayor valor de esta métrica y en azul aquellos con menor valor.

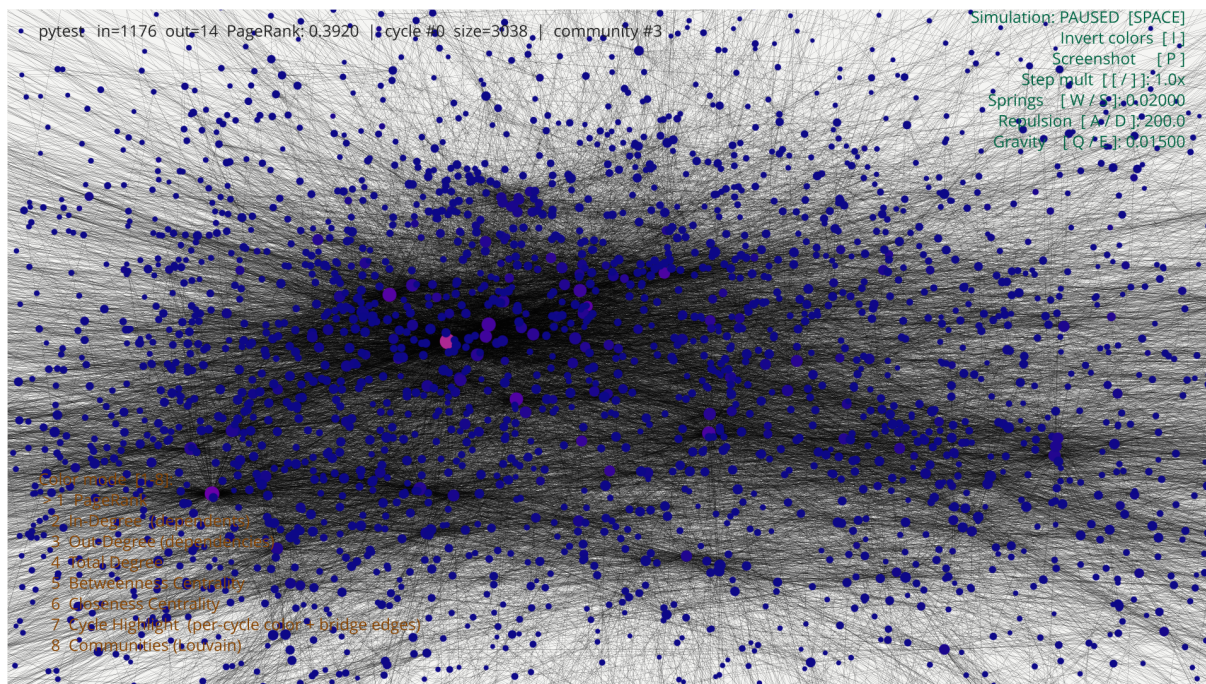


Figura 2.12: Representación del PageRank en la red con zoom de dependencias de Python donde en amarillo están los paquetes con mayor valor de esta métrica y en azul aquellos con menor valor.

Aun así, al analizar estos paquetes que han salido con los valores más elevados de este métrica de centralidad se confirma que esta es adecuada para el análisis de importancia pues los paquetes que salen arriba del todo son fundamentales. `typing-extensions` es una librería utilizada para forzar la definición explícita de los tipos de datos en Python. De esta manera, se soluciona el problema fundamental de los errores de tipo en el intérprete, lo que da sentido a que sea uno de los paquetes más relevantes del ecosistema. `pytest` el segundo de la lista es un paquete también esencial para el desarrollo porque es una herramienta necesaria en proyectos grandes para poder realizar los test del código. `numpy` es el tercero de la lista y es el paquete para realizar álgebra de manera eficiente en Python por lo que es lógico que tenga mucha relevancia porque la mayoría de paquetes necesitan realizar operaciones eficientes. Por tanto, el PageRank es una buena medida para evaluar como de útil es un paquete dentro del ecosistema de desarrollo.

Asimismo, es necesario recalcar el problema de la duplicidad de nodos introducido por cómo se ha construido el grafo. Tal y como se evidenció previamente con el paquete `ansible`, las inconsistencias en la nomenclatura utilizada por los desarrolladores provocan que una librería pueda no ser encontrada o crear entradas duplicadas. Esto implica que los paquetes más relevantes del ecosistema concentran, en realidad, un *PageRank* aún mayor del que reflejan sus métricas individuales actuales. Ejemplos claros de esta división se observan en las entradas separadas para `typing-extensions` y `typing_extensions`, o `sphinx` y `Sphinx`. Para resolver esta problemática, en PyPi probablemente se utilicen herramientas que conviertan todos los nombres a texto plano y de esta forma se eliminen estas inconsistencias.

Por otro lado, cabe destacar que, excluyendo a `numpy` y `requests` (que son paquetes de uso directo para aplicaciones muy específicas), la mayor parte del top 10 está conformada por paquetes necesarios para mantener proyectos complejos. En consecuencia, se evidencia que los paquetes más importantes de la red representan la infraestructura clave que permite al resto de herramientas tener un ciclo de desarrollo más ágil y escalable.

Finalmente, resulta llamativo encontrar en la quinta posición a `pyobjc-core`, una librería exclusiva para la integración con sistemas operativos macOS. Dado que el ecosistema de desarrollo en Python está fuertemente dominado por entornos Linux y Windows, la cuota de mercado de macOS no justificaría por sí sola una centralidad tan elevada. Sin embargo, esta anomalía ilustra perfectamente la naturaleza de las dependencias condicionales en proyectos multiplataforma. Grandes paquetes de automatización o interfaces gráficas incluyen a `pyobjc-core` como dependencia oculta, instalándose automáticamente solo cuando el código se ejecuta en un entorno de Apple. Así, la librería acumula un *PageRank* masivo de forma pasiva.

2.2.2. Centralidad de Intermediación

La centralidad de intermediación mide la frecuencia con la que un nodo está en el camino más corto entre otros dos nodos del grafo. En el contexto del ecosistema Python, esta métrica indica que paquetes conectan paquetes de diferentes ecosistemas o que funcionan de interfaces estandarizadoras de los datos. Visualmente en las figuras 2.13 y 2.14, se puede ver que a diferencia del PageRank existe una mayor homogeneidad en cuanto a esta métrica de centralidad, es decir, ya no existen tantos paquetes críticos.

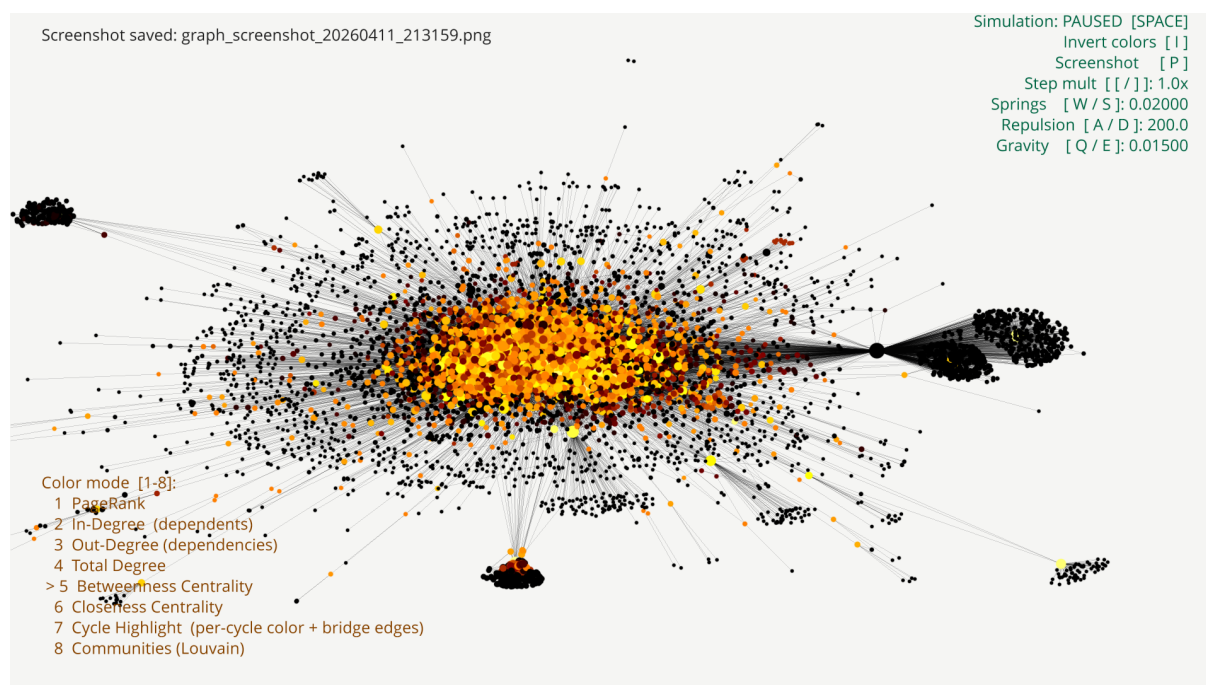


Figura 2.13: Representación de la Centralidad de Intermediación en la red completa de dependencias de Python donde en blanco y amarillo están los paquetes con mayor valor de esta métrica y en negro y rojo aquellos con menor valor.

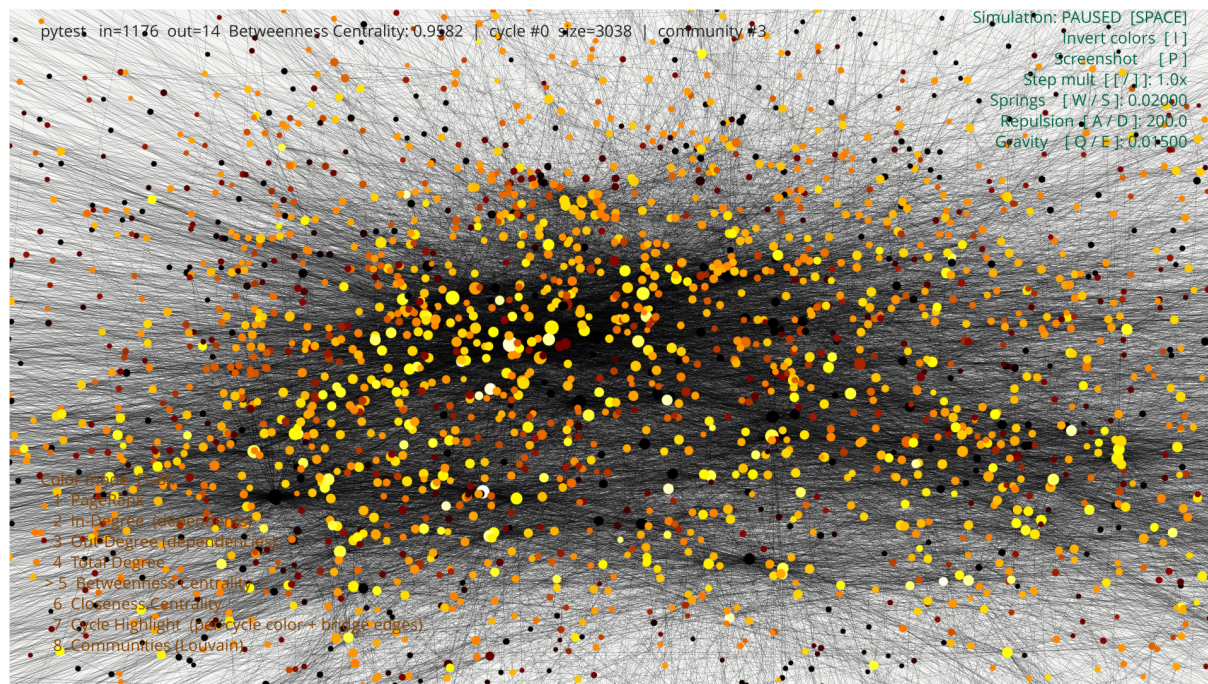


Figura 2.14: Representación de la Centralidad de Intermediación en la red con zoom de dependencias de Python donde en blanco y amarillo están los paquetes con mayor valor de esta métrica y en negro y rojo aquellos con menor valor.

Al examinar las primeras posiciones de esta métrica, se comprueba que refleja fielmente la función de estos paquetes como puentes del ecosistema. **pandas** ocupa el primer lugar al ser el estándar de facto para la manipulación de datos tabulares. Por tanto, su integración es un requisito para casi cualquier librería orientada al análisis de datos. A continuación, la segunda posición de **pyiceberg** subraya el inmenso peso que tiene el procesamiento de grandes volúmenes de información en Python en formato tabular. Dada su alta centralidad de intermediación, la adopción masiva de estas herramientas sugiere que la manipulación de datos tabulares es tan esencial dentro del ecosistema Python que podría motivar su futura integración como un tipo nativo en el estándar de Python.

A continuación, destaca la posición de **ipython**, que actúa como nexo fundamental entre los entornos de programación interactiva como Jupyter y las herramientas de depuración del lenguaje. Esta librería funciona como un puente estructural, conectando a la vasta comunidad centrada en el análisis exploratorio con el núcleo de desarrollo tradicional de Python. Por otro lado, la presencia de **hypothesis** y **pytest** cerrando el *top 5* revela la transversalidad de las pruebas de software. Independientemente del subdominio al que pertenezca un paquete, el uso de estos frameworks es un estándar para garantizar la calidad y compatibilidad del código, convirtiéndolos en puntos que unen todas las ramas de la red.

2.2.3. Centralidad de Cercanía

La centralidad de cercanía evalúa la distancia media desde un nodo hacia todos los demás nodos de la red. Desde una perspectiva de arquitectura, esto implica que cualquier

cambio en estos nodos centrales se propagará rápidamente a lo largo de toda la red. Visualmente, tal y como se observa en las figuras 2.15 y 2.16, la red presenta una apariencia densa y comprimida donde resulta complejo distinguir variaciones significativas entre los nodos. Esta homogeneidad visual se debe a la componente no conexa del paquete `ansible` que provoca una distorsión que aplasta los valores de centralidad de la inmensa mayoría de la red principal en torno al 0,4. Para mitigar este efecto y poder extraer conclusiones analíticas válidas este paquete se ha sido excluido del ranking numérico analizado a continuación.

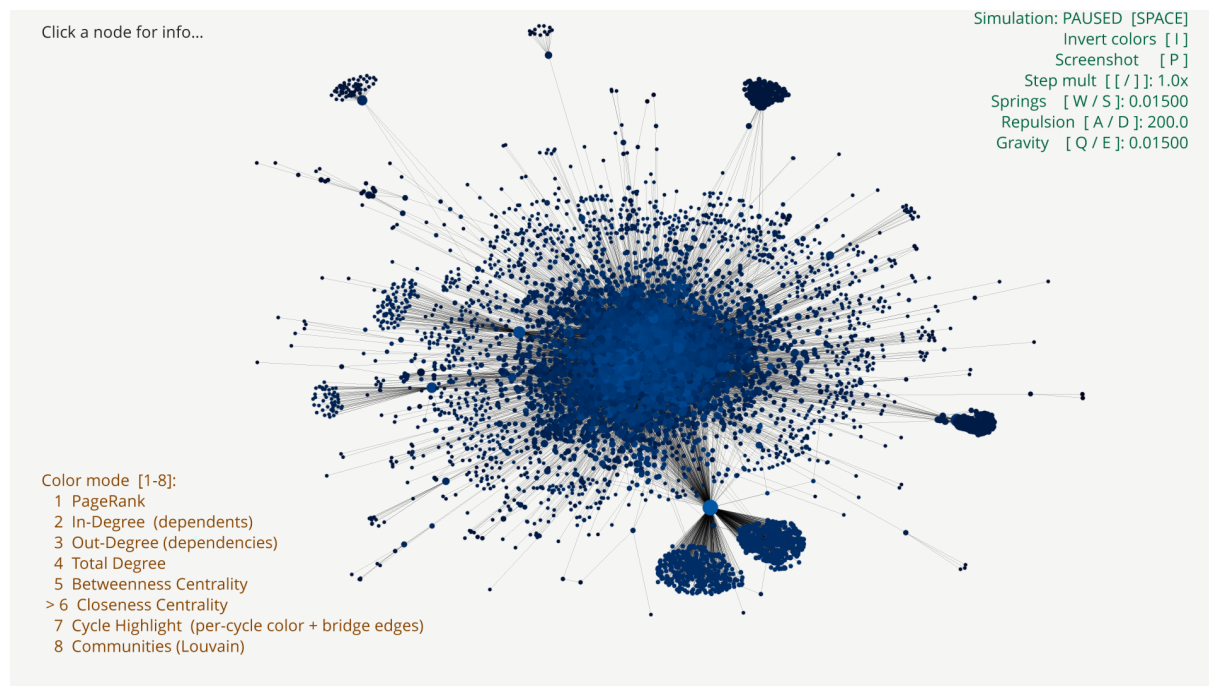


Figura 2.15: Representación de la Centralidad de Cercanía en la red completa de dependencias de Python donde en blanco y cian están los paquetes con mayor valor de esta métrica y en azul oscuro aquellos con menor valor.

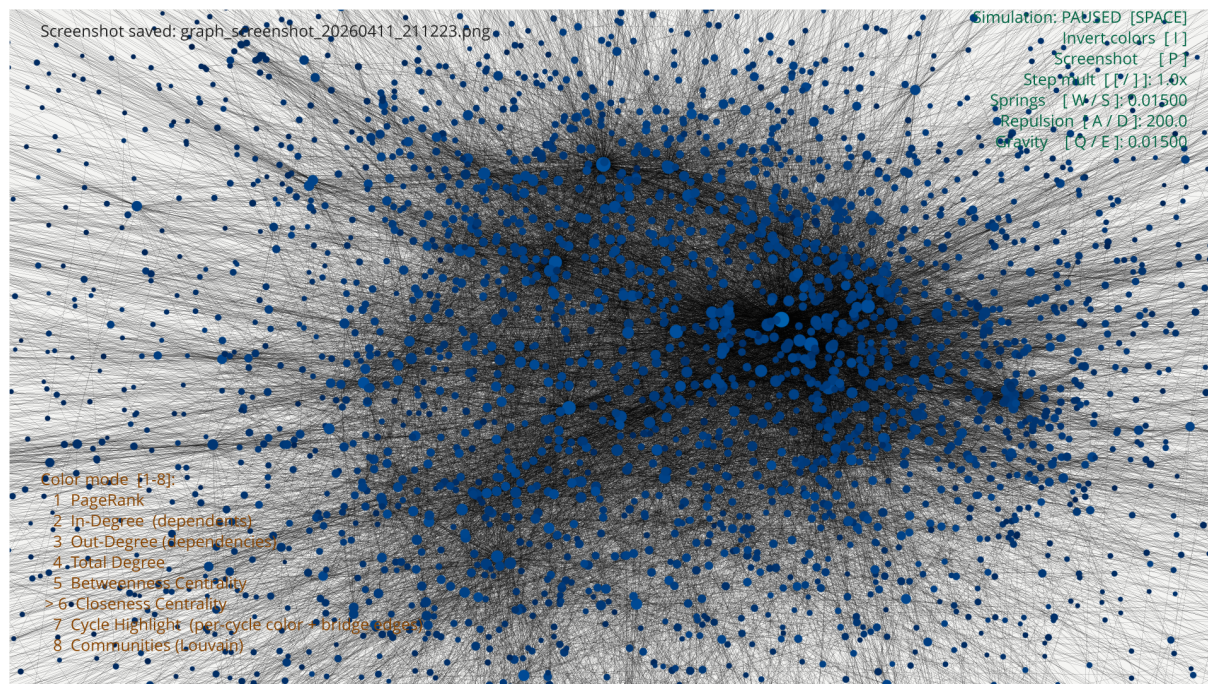


Figura 2.16: Representación de la Centralidad de Cercanía en la red con zoom de dependencias de Python donde en blanco y cian están los paquetes con mayor valor de esta métrica y en azul oscuro aquellos con menor valor.

Al examinar los datos depurados y normalizados, no se observan grandes diferencias entre los paquetes y parecen estar distribuidos de manera muy uniforme. No obstante, las conclusiones de centralidad que se obtienen son semejantes a las obtenidas mediante las métricas de PageRank y de centralidad de intermediación.

2.2.4. Distribuciones de Centralidad

Para comprender la topología de la red de dependencias, es fundamental analizar la distribución de sus métricas de centralidad. Al observar las figuras 2.17 y 2.18, se identifican tres patrones geométricos distintos que responden a leyes matemáticas específicas.

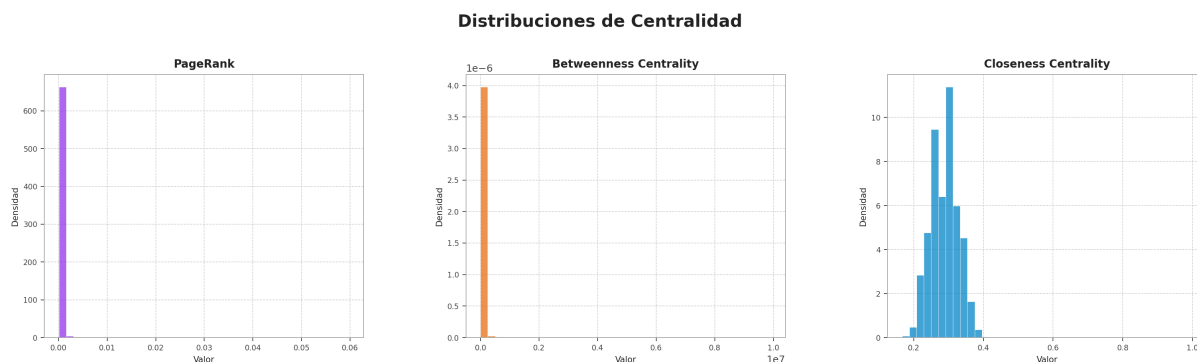


Figura 2.17: Distribución de frecuencias de las métricas de centralidad en escala lineal. Se observa una asimetría extrema en la mayoría de variables.

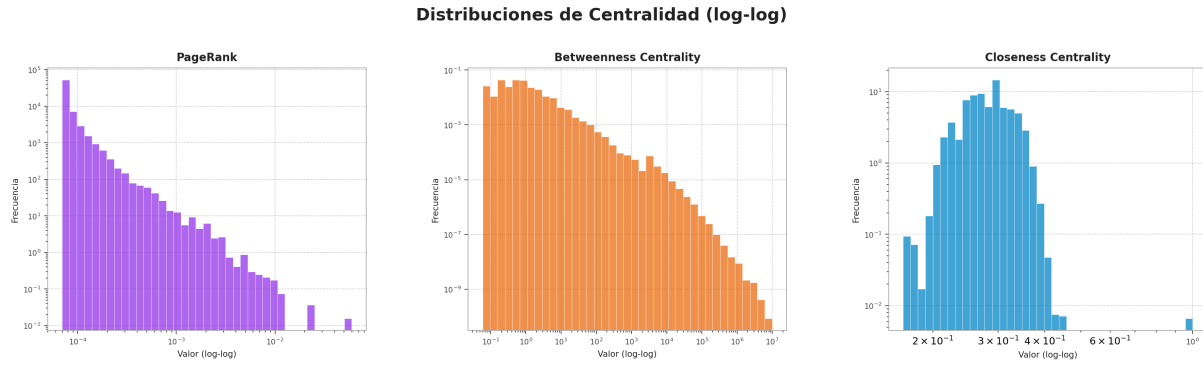


Figura 2.18: Distribución de frecuencias de las métricas de centralidad en escala Log-Log, revelando las leyes de potencias y decaimientos estructurales.

PageRank

En la escala Log-Log, esta métrica se distribuye siguiendo una línea recta descendente. Este comportamiento es característico de las redes libres de escala, donde la función sigue la relación:

$$P(PR) \sim PR^{-\gamma} \implies \log(P(PR)) = -\gamma \log(PR) + C \quad (2.13)$$

Este comportamiento muestra que los paquetes más importantes siguen creciendo y siendo cada vez más importantes, es decir, un escenario de rich get richer. Dentro de un ecosistema de paquetes esto es lógico puesto que existen dependencias que por la función esencial que desempeñan acaban siendo los más importantes.

Centralidad de Intermediación

A diferencia de la anterior, la centralidad de intermediación no forma una recta, sino una curva que cae de forma cuadrática en la escala logarítmica. Esta relación tiene una ecuación de tipo:

$$\log(P(CI)) \approx -K(\log CI)^2 + C \quad (2.14)$$

Esta curvatura indica que ser un puente es un rol mucho más exclusivo que ser popular.

Centralidad de Cercanía

La centralidad de cercanía rompe con las asimetrías anteriores y presenta una distribución similar a una distribución normal:

$$P(CC) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{CC-\mu}{\sigma}\right)^2} \quad (2.15)$$

Este fenómeno se debe a la existencia del efecto Pequeño Mundo.

2.3. Análisis de comunidades

Para analizar las comunidades se utiliza el algoritmo de Louvain aplicado sobre el grado no dirigido. El análisis de esta sección queda reflejado mediante la figura 2.19 y la tabla 2.4 generada mediante el script `communities.py`.

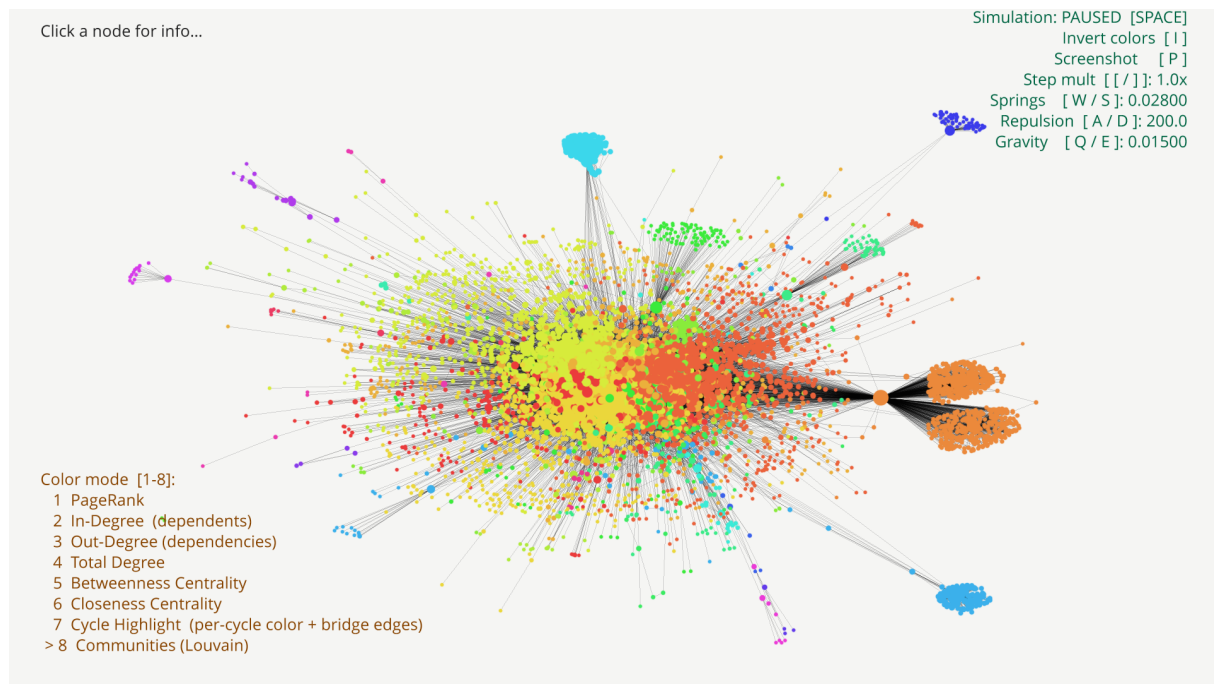


Figura 2.19: Representación visual de las comunidades obtenidas en el grafo de la red.

Tabla 2.4: Descripción de las comunidades de la red de dependencias de Python.

ID	Nodos	Descripción
0	413	Esta comunidad agrupa el ecosistema de computación interactiva basado en <code>jupyter</code> , <code>jupyterlab</code> , <code>notebook</code> e <code>ipython</code> , junto con su infraestructura de soporte (<code>ipykernel</code> , <code>jupyter-server</code> , <code>nbconvert</code>). Se complementa con un extenso conjunto de herramientas de documentación técnica centradas en <code>sphinx</code> y sus extensiones (<code>myst-parser</code> , <code>pydata-sphinx-theme</code> , <code>nbsphinx</code> , <code>sphinx-copybutton</code>), parsers de Markdown (<code>markdown-it-py</code>), e interfaces gráficas Qt (<code>PyQt5</code> , <code>qtconsole</code>). Incluye además herramientas de pruebas para cuadernos (<code>nbval</code> , <code>pytest-jupyter</code>) e interoperabilidad con R mediante <code>rpy2</code> .
1	1197	Comunidad centrada en el desarrollo de APIs web y servicios backend modernos, con <code>fastapi</code> , <code>starlette</code> , <code>pydantic</code> y <code>uvicorn</code> como núcleo principal. Incluye clientes HTTP (<code>httpx</code> , <code>requests</code> , <code>aiohttp</code>), integración con servicios cloud (<code>boto3</code> , <code>azure-identity</code> , <code>google-cloud-aiplatform</code>), mensajería y colas (<code>celery</code> , <code>kombu</code> , <code>redis</code>), bases de datos (<code>sqlalchemy</code> , <code>pymongo</code>), y un extenso ecosistema de inteligencia artificial generativa con <code>langchain</code> , <code>openai</code> , <code>anthropic</code> , <code>llama-index</code> y <code>pydantic-ai</code> . También abarca herramientas de observabilidad (<code>opentelemetry-sdk</code>), autenticación (<code>cryptography</code> , <code>pyjwt</code>) y orquestación de flujos (<code>prefect</code> , <code>langgraph</code>).

Continúa en la página siguiente

Tabla 2.4: Descripción de las comunidades de la red de dependencias de Python. (Continuación)

2	867	Comunidad dominada por definiciones de tipos estáticos para el ecosistema AWS, con miles de paquetes <code>types-boto3-*</code> y <code>types-aiobotocore-*</code> que proveen anotaciones de tipo para cada servicio de Amazon Web Services. Complementariamente, incluye herramientas de serialización y modelado de documentación de código fuente como <code>databind.core</code> , <code>databind.json</code> , <code>docspec</code> y <code>docspec-python</code> , así como utilidades de análisis de grafos de importación (<code>grimp</code>) y logging tipado (<code>loguru-mypy</code>). Esta comunidad es esencial para el desarrollo de aplicaciones Python con tipado estático sobre infraestructura AWS.
3	641	Comunidad orientada al análisis y manipulación de datos tabulares y geoespaciales. Su núcleo lo forman <code>pandas</code> , <code>numpy</code> , <code>polars</code> , <code>dask</code> y <code>xarray</code> , junto con conectores a bases de datos y formatos de archivo (<code>sqlalchemy</code> , <code>pyarrow</code> , <code>openpyxl</code> , <code>fastparquet</code> , <code>duckdb</code>). Incluye herramientas de visualización (<code>plotly</code> , <code>bokeh</code> , <code>plotnine</code> , <code>dash</code>), análisis geoespacial (<code>geopandas</code> , <code>shapely</code> , <code>fiona</code> , <code>rasterio</code>), astronomía (<code>astropy</code> , <code>skyfield</code>), y frameworks web ligeros para datos (<code>shiny</code> , <code>flask</code>). También abarca motores de consulta distribuida (<code>pyspark</code> , <code>ibis-framework</code>) y utilidades de grafos (<code>neo4j</code> , <code>networkx</code>).
4	1128	Comunidad dedicada a las herramientas de calidad del software, empaquetado y desarrollo de aplicaciones Python. Agrupa frameworks de pruebas (<code>pytest</code> , <code>pytest-cov</code> , <code>tox</code>), linters y formateadores (<code>flake8</code> , <code>black</code> , <code>isort</code> , <code>ruff</code>), verificación de tipos (<code>mypy</code> , <code>pyright</code>), y generación de documentación (<code>sphinx</code> , <code>sphinx-rtd-theme</code>). Incluye herramientas de empaquetado y distribución (<code>setuptools</code> , <code>build</code> , <code>flit</code> , <code>hatchling</code> , <code>twine</code>), frameworks de aplicaciones GUI (<code>kivy</code>) y web (<code>django</code> , <code>flask</code> , <code>sanic</code>), además de utilidades de serialización (<code>marshmallow</code>), análisis estático (<code>pylint</code> , <code>vulture</code>) y herramientas de CI/CD (<code>dvc</code> , <code>pre-commit</code>).
5	1169	Gran comunidad centrada en el aprendizaje automático y la inteligencia artificial. Tiene como núcleo <code>tensorflow</code> , <code>torch</code> (PyTorch), <code>jax</code> y <code>keras</code> , junto con librerías científicas de soporte como <code>numpy</code> , <code>scipy</code> , <code>scikit-learn</code> y <code>matplotlib</code> . Abarca visión artificial (<code>torchvision</code> , <code>opencv-python</code> , <code>albumentations</code>), procesamiento de lenguaje natural (<code>transformers</code> , <code>tokenizers</code> , <code>sentence-transformers</code> , <code>nltk</code>), modelos de difusión (<code>diffusers</code>), optimización (<code>optuna</code> , <code>botorch</code>), y aceleración en GPU (<code>nvidia-cublas-cu12</code> , <code>triton</code>). También incluye herramientas de evaluación (<code>evaluate</code> , <code>lm-eval</code>), despliegue de modelos (<code>onnxruntime</code> , <code>vllm</code>) y frameworks especializados (<code>lightning</code> , <code>deepspeed</code>).

Continúa en la página siguiente

Tabla 2.4: Descripción de las comunidades de la red de dependencias de Python. (Continuación)

6	179	Comunidad enfocada en computación de alto rendimiento con GPU y verificación formal de software. Agrupa librerías CUDA y RAPIDS de NVIDIA (<code>cudf-cu12</code> , <code>cupy-cuda12x</code> , <code>cuda-bindings</code> , <code>numba-cuda</code>), herramientas de pruebas basadas en propiedades (<code>hypothesis</code> , <code>hypothesis-crosshair</code>), análisis numérico (<code>mpmath</code> , <code>scipy-stubs</code>), y el ecosistema de procesamiento de lenguaje natural industrial <code>spacy</code> con sus componentes (<code>thinc</code> , <code>blis</code> , <code>cymem</code>). También incluye herramientas de la pila OpenStack (<code>oslo.config</code> , <code>oslo.utils</code> , <code>stevedore</code>) y cobertura de código (<code>codecov</code>).
7	157	Comunidad centrada en el ecosistema de Google Cloud Platform y sus APIs. Agrupa clientes oficiales para servicios como <code>google-cloud-bigquery</code> , <code>google-cloud-storage</code> , <code>google-cloud-pubsub</code> , <code>google-cloud-spanner</code> y <code>google-cloud-aiplatform</code> , junto con la infraestructura de autenticación y comunicación (<code>grpcio</code> , <code>google-auth</code> , <code>proto-plus</code> , <code>googleapis-common-protos</code>). Incluye además el framework de pipelines de ML <code>kfp</code> (Kubeflow Pipelines), clientes de Apache Beam para procesamiento distribuido, y herramientas criptográficas de Google (<code>tink</code>).
8	2	Comunidad no conexas compuesta exclusivamente por <code>ansible</code> y <code>ansible-core</code> . Esta comunidad se ha generado por un problema al parsear los nombres de los paquetes.
9	205	Comunidad orientada a la documentación con <code>mkdocs</code> y sus extensiones (<code>mkdocs-material</code> , <code>mkdocstrings</code> , <code>mkdocs-gen-files</code> , <code>mkdocs-jupyter</code>), como alternativa moderna a Sphinx. Incluye el framework de arrays funcional <code>equinox</code> y <code>jaxtyping</code> para JAX, herramientas de análisis de datos con <code>ibis-framework</code> y <code>kedro</code> para pipelines de datos reproducibles, así como conectores a múltiples backends (<code>kedro-datasets</code>). También abarca formatos de datos dispersos (<code>sparse</code> , <code>zarr</code>) y herramientas de series temporales (<code>u8darts</code>).
10	46	Comunidad centrada en el scraping web y la transformación de datos. Incluye <code>scrapy</code> como framework principal de extracción web, junto con <code>beautifulsoup4</code> , <code>newspaper3k</code> y <code>requests-html</code> para el análisis de contenido HTML. Agrupa además el ecosistema <code>dbt</code> (<code>dbt-core</code> , <code>dbt-postgres</code> , <code>dbt-bigquery</code> , <code>dbt-snowflake</code>) para transformación de datos en almacenes SQL, y herramientas de orquestación asociadas (<code>prefect-dbt</code> , <code>prefect-sqlalchemy</code>).

Continúa en la página siguiente

Tabla 2.4: Descripción de las comunidades de la red de dependencias de Python. (Continuación)

11	115	Comunidad agrupada en torno a <code>ruamel.yaml</code> y sus extensiones para el procesamiento avanzado de YAML, junto con servidores web de alto rendimiento como <code>granian</code> y <code>uvloop</code> . Incluye el framework de MLOps <code>mlrun</code> con sus numerosas dependencias versionadas, herramientas de validación de especificaciones OpenAPI (<code>prance</code> , <code>openapi-spec-validator</code>), y clientes para servicios cloud de Alibaba (<code>aliyun-python-sdk-core</code> , <code>oss2</code>). También contiene <code>deepldiff</code> para comparación de estructuras de datos.
12	5	Comunidad mínima formada por el binding oficial de Qt para Python mediante <code>PySide6</code> , <code>shiboken6</code> , <code>PySide6_Essentials</code> y <code>PySide6_Addons</code> , representando la alternativa LGPL a PyQt5 para el desarrollo de interfaces gráficas de escritorio con Qt6.
13	28	Comunidad centrada en el análisis sintáctico y la representación estructurada de texto. Agrupa <code>tree-sitter</code> y sus gramáticas para múltiples lenguajes (<code>tree-sitter-python</code> , <code>tree-sitter-javascript</code> , <code>tree-sitter-rust</code> , entre otros), utilizados para el análisis de código fuente. Incluye además <code>textual</code> para interfaces de terminal, <code>docling-core</code> y <code>docling-parse</code> para extracción de documentos, y <code>latex2mathml</code> para conversión de fórmulas matemáticas.
14	246	Comunidad dominada por el AWS Cloud Development Kit (<code>aws-cdk-lib</code> y la familia <code>aws-cdk.aws-*</code>), que proporciona abstracciones de infraestructura como código para prácticamente todos los servicios de Amazon Web Services. El núcleo lo forman <code>constructs</code> , <code>jsii</code> y <code>aws-cdk.core</code> , con módulos especializados para servicios como <code>aws-cdk.aws-lambda</code> , <code>aws-cdk.aws-ec2</code> , <code>aws-cdk.aws-s3</code> , <code>aws-cdk.aws-rds</code> y <code>aws-cdk.aws-eks</code> , entre muchos otros.
15	220	Comunidad centrada en el framework de red asíncrono <code>twisted</code> y el extenso ecosistema de bindings <code>pyobjc</code> para macOS. El componente <code>twisted</code> incluye soporte para protocolos de red, TLS y concurrencia (<code>automat</code> , <code>hyperlink</code> , <code>incremental</code>), mientras que <code>pyobjc</code> expone prácticamente todos los frameworks nativos de Apple (<code>pyobjc-framework-Cocoa</code> , <code>pyobjc-framework-Metal</code> , <code>pyobjc-framework-CoreML</code> , entre otros). También incluye <code>rinohype</code> para composición tipográfica de documentos PDF y <code>mido</code> para el manejo de datos MIDI.
16	4	Comunidad mínima formada por <code>Pyro4</code> y sus dependencias directas (<code>serpent</code> , <code>selectors34</code>), que implementan un sistema de objetos remotos para Python, permitiendo la comunicación transparente entre procesos distribuidos.

Continúa en la página siguiente

Tabla 2.4: Descripción de las comunidades de la red de dependencias de Python. (Continuación)

17	5	Comunidad pequeña agrupada en torno a utilidades de logging con color y formato legible para la terminal, representada por <code>coloredlogs</code> y <code>humanfriendly</code> , junto con <code>capturer</code> para captura de salida estándar, y compatibilidades de bajo nivel (<code>monotonic</code> , <code>pyreadline</code>).
18	64	Comunidad formada íntegramente por el framework multiagente <code>ag2</code> (evolución de AutoGen) y sus numerosas variantes de instalación opcionales (<code>ag2[openai]</code> , <code>ag2[anthropic]</code> , <code>ag2[langchain]</code> , <code>ag2[rag]</code> , <code>ag2[browser-use]</code> , entre otras), que cubren integraciones con distintos proveedores de modelos de lenguaje, bases de datos vectoriales, herramientas de búsqueda y capacidades multimodales para la construcción de sistemas de agentes autónomos.
19	4	Comunidad mínima compuesta por <code>mitmproxy-rs</code> y sus variantes específicas por plataforma (<code>mitmproxy-windows</code> , <code>mitmproxy-linux</code> , <code>mitmproxy-macos</code>), que proporcionan los componentes nativos en Rust para el proxy HTTP/HTTPS interactivo <code>mitmproxy</code> .
20	4	Comunidad mínima formada por el binding de Qt6 para Python <code>PyQt6</code> y sus componentes asociados (<code>PyQt6-sip</code> , <code>PyQt6-Qt6</code>), representando la versión más reciente del framework para el desarrollo de interfaces gráficas de escritorio multiplataforma con PyQt.
21	18	Comunidad compuesta por librerías de bajo nivel de NVIDIA para computación en GPU con CUDA 13, incluyendo componentes como <code>nvidia-cublas</code> , <code>nvidia-cudnn-cu13</code> , <code>nvidia-cuda-runtime</code> , <code>nvidia-nccl-cu13</code> y <code>nvidia-nvshmem-cu13</code> , junto con el plugin <code>jax-cuda13-plugin</code> para la integración de JAX con esta generación de CUDA.
22	14	Comunidad formada por librerías de bajo nivel del ecosistema ROCm de AMD para computación en GPU, incluyendo <code>amd_rocm_hip_runtime_instinct</code> , <code>amd_hipblas_instinct</code> , <code>amd_miopen_hip_instinct</code> y <code>amd_rccl_instinct</code> , junto con el plugin <code>jax-rocm7-plugin</code> para la integración de JAX con hardware GPU de AMD.
23	9	Comunidad pequeña formada por el SDK nativo de Python para Amazon Bedrock (<code>aws-sdk-bedrock-runtime</code>) y la pila de librerías Smithy (<code>smithy-core</code> , <code>smithy-http</code> , <code>smithy-aws-core</code> , <code>smithy-json</code>), que implementan el framework de modelado de servicios de AWS para la generación y consumo de APIs cloud de nueva generación.

Continúa en la página siguiente

Tabla 2.4: Descripción de las comunidades de la red de dependencias de Python. (Continuación)

24	10	Comunidad centrada en el desarrollo de interfaces de usuario interactivas en la terminal. Agrupa <code>inquirer</code> para menús de selección interactivos, <code>blessed</code> y <code>jinxed</code> para el control de terminal multiplataforma, <code>readchar</code> para la lectura de teclas, y extensiones de Sphinx como <code>sphinx-paramlinks</code> y <code>sphinxcontrib-manpage</code> para documentación de parámetros y páginas de manual.
25	7	Comunidad mínima formada por el ecosistema <code>itkwasm</code> para el procesamiento de imágenes médicas en WebAssembly, incluyendo <code>itkwasm-image-io</code> , <code>itkwasm-downsample</code> y sus variantes para los entornos de ejecución Emscripten y WASI, que permiten ejecutar algoritmos de la librería ITK directamente en el navegador o en entornos WASM.
26	8	Comunidad mínima centrada en la verificación matemática formal, compuesta por <code>math-verify</code> para la comprobación automática de soluciones matemáticas y <code>latex2sympy2_extended</code> para la conversión de expresiones $\text{\LaTeX}$ a objetos simbólicos de SymPy, con variantes para distintas versiones de ANTLR4 (<code>antlr4_9_3</code> , <code>antlr4_11_0</code> , <code>antlr4_13_2</code>). Se utiliza principalmente en el contexto de evaluación de modelos de lenguaje en tareas matemáticas (<code>lighteval[math]</code>).

Capítulo 3

Herramienta de visualización de la red

Para poder explorar de forma interactiva la red de dependencias de Python se desarrolla una herramienta de visualización en tiempo real basada en un motor de físicas implementado íntegramente en Python. La herramienta, recogida en el script `visualizer.py`. Mediante esta aplicación se puede navegar a través de la red, inspeccionar diferentes métricas en función del nodo y ver como va variando la estructura de la red en función de las físicas escogidas con mejor rendimiento para redes de este tamaño que en Gephi.

3.1. Arquitectura general

El diseño de la herramienta se basa en las siguientes librerías principalmente:

- **igraph**: se utiliza para cargar el grafo desde el fichero `pypi_multiseed_10k.graphml` y del cálculo de la mayoría de métricas relacionadas con grafos vistas en clase.
- **vispy**: se utiliza como motor de renderizado interactivo que hace uso de OpenGL mediante aceleración por GPU y para capturar los inputs de teclado y ratón.
- **numpy**: se utiliza para la vectorización de todas las operaciones del motor de físicas y del cómputo de colores por nodo de manera eficiente.

La resolución del monitor se consulta en tiempo de ejecución a través de `glfw` antes de crear la ventana porque para que se vea adecuadamente la leyenda es necesario trabajar a pantalla completa. Se ha utilizado esta librería para el renderizado porque entre las distintas librerías dentro de `vispy` es la que mejor rendimiento ha dado en el equipo utilizado.

3.2. Cálculos preliminares

Para calcular una disposición inicial “razonable” de los nodos de manera eficiente, se emplea el algoritmo *DrL* (Distributed Recursive Layout) integrado en la librería **igraph**. Aunque los reportes técnicos originales de este método ya no son de acceso público, su

funcionamiento se encuentra documentado en el artículo correspondiente un algoritmo que lo mejora, la herramienta OpenOrd [2]. En la figura 3.1 se puede ver la distribución inicial que genera este algoritmo.

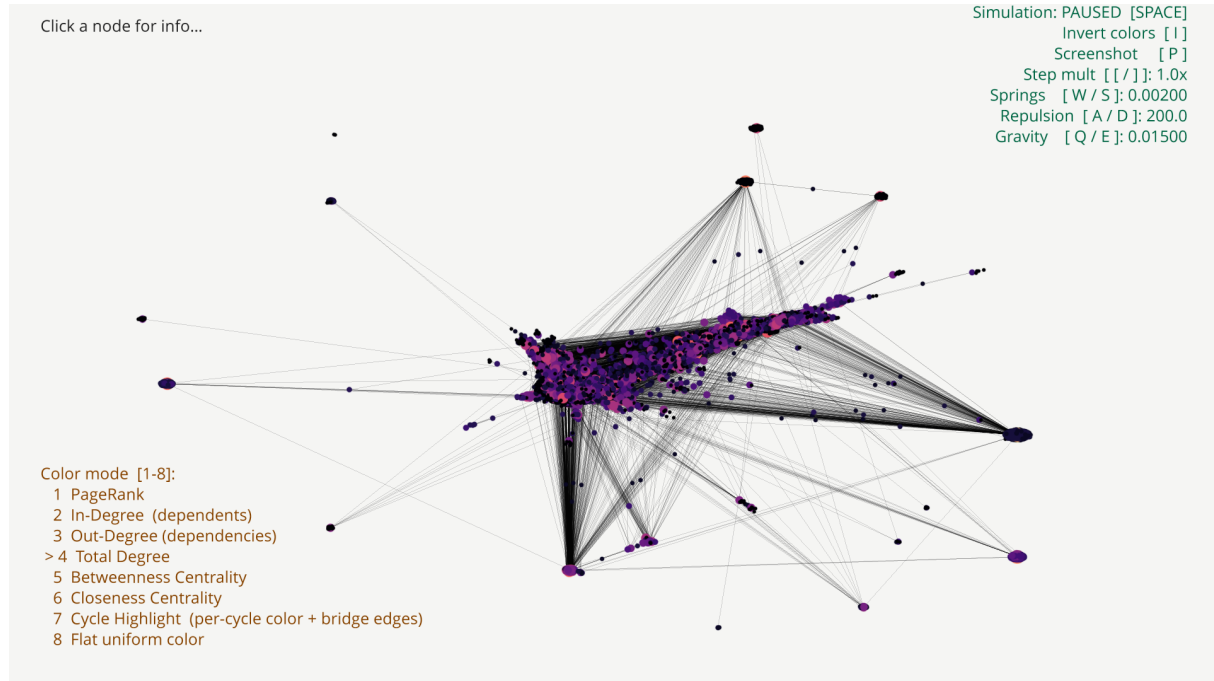


Figura 3.1: Representación visual de la red tras aplicar el algoritmo DrL.

El tamaño de cada nodo se determina mediante una normalización logarítmica del grado total para que los nodos de mayor tamaño sean más visibles, pero dejando un tamaño mínimo de 4 píxeles.

$$s_i = 21 \cdot \hat{G}_i + 4, \quad \hat{G}_i = \frac{\log(1 + G_i^{\text{in}} + G_i^{\text{out}})}{\max_j \log(1 + G_j^{\text{in}} + G_j^{\text{out}})} \quad (3.1)$$

donde s_i es el tamaño del nodo y G es el grado de entrada/salida de cada nodo.

En cuanto a las aristas para que los nodos se pudieran visualizar correctamente, estas se pintan “por debajo” en el lienzo por lo que las flechas de dirección no son fácilmente visibles. No obstante, como se puede alternar el esquema de colores para ver los grados de entrada y salida, al igual que hacer click en ellos, se puede saber fácilmente la direccionalidad de las flechas a nivel macroscópico. Además, para ver mejor la concentración de aristas cuando estas se superponen se reduce su transparencia por lo que se ve su color más marcado.

3.3. Motor de físicas

Para hacer la visualización más intuitiva del grafo en la herramienta se incorpora un motor de físicas basado en fuerzas que permite reorganizar el grafo en tiempo real. La

simulación puede activarse y reanudarse con la tecla `ESPACIO`, y su velocidad se controla mediante un multiplicador ajustable. En cada fotograma (`on_timer_tick`), la fuerza neta $\mathbf{F}_i = \mathbf{F}_{H,i} + \mathbf{F}_{g,i} + \mathbf{F}_{r,i}$ aplicada sobre cada nodo i con posición $\mathbf{p}_i$ (`pos`) se calcula como la suma de tres fuerzas principales:

- **Fuerza de Hooke:** las aristas actúan como muelles que atraen los nodos conectados entre sí. Para un nodo i , la fuerza ejercida por el conjunto de sus nodos adyacentes $\mathcal{N}(i)$ se define como:

$$\mathbf{F}_{H,i} = \sum_{j \in \mathcal{N}(i)} k_s (\mathbf{p}_j - \mathbf{p}_i)$$

donde k_s es `spring_strength`. En la implementación, esto se vectoriza calculando la diferencia direccional ($\mathbf{p}_1 - \mathbf{p}_0$) y aplicando la fuerza simétricamente en ambos extremos de la arista.

- **Fuerza gravitatoria:** atrae a cada nodo hacia el origen de coordenadas $(0,0)$ con una intensidad proporcional a su grado total, manteniendo el grafo centrado y evitando la dispersión indefinida de los nodos más periféricos:

$$\mathbf{F}_{g,i} = -k_g G_i \mathbf{p}_i$$

donde k_g es `gravity` y G_i corresponde a su grado normalizado (`norm_degrees_2d`).

- **Fuerza de repulsión:** para que el programa no se congele al calcular la fuerza de repulsión de manera tradicional se calcula en cada frame las fuerzas de repulsión para un subconjunto $\mathcal{R}$ de tamaño $N_R = 300$ (`sample_size`). De esta manera, aunque el calculo no es del todo exacto, se agilizan los cálculos y se asume que la distribución visual final no variaría demasiado en el equilibrio:

$$\mathbf{F}_{r,i} = \sum_{j \in \mathcal{R}} k_r \frac{\mathbf{p}_i - \mathbf{p}_j}{\|\mathbf{p}_i - \mathbf{p}_j\|^2 + 1}$$

donde k_r es `outward_push`. Se añade un $+1$ en el denominador para evitar singularidades en el caso de que dos nodos estén superpuestos. Se utiliza esta formula de la fuerza para que los nodos tengan más alcance en la repulsión para reducir las aglomeraciones de nodos.

A partir de las fuerzas se calcula la velocidad en cada frame asumiendo un $\delta t = 1$ y masa unitaria mediante integración numérica con un coeficiente de amortiguamiento μ caracterizado por la variable `"friction"` y limitada por una velocidad máxima (`"max_speed"`).

$$\mathbf{v}_i^{(t+1)} = \left(\mathbf{v}_i^{(t)} + \mathbf{F}_{\text{total},i} \right) \cdot \mu$$

Por último, se calculan las posiciones de manera análoga a las velocidad mediante integración discreta:

$$\mathbf{p}_i^{(t+1)} = \mathbf{p}_i^{(t)} + \mathbf{v}_i^{(t+1)}$$

De esta manera se puede ver como queda el resultado de aplicar el algoritmo tal y como se puede ver en la figura 3.2 en comparación con el estado inicial en la figura 3.1.

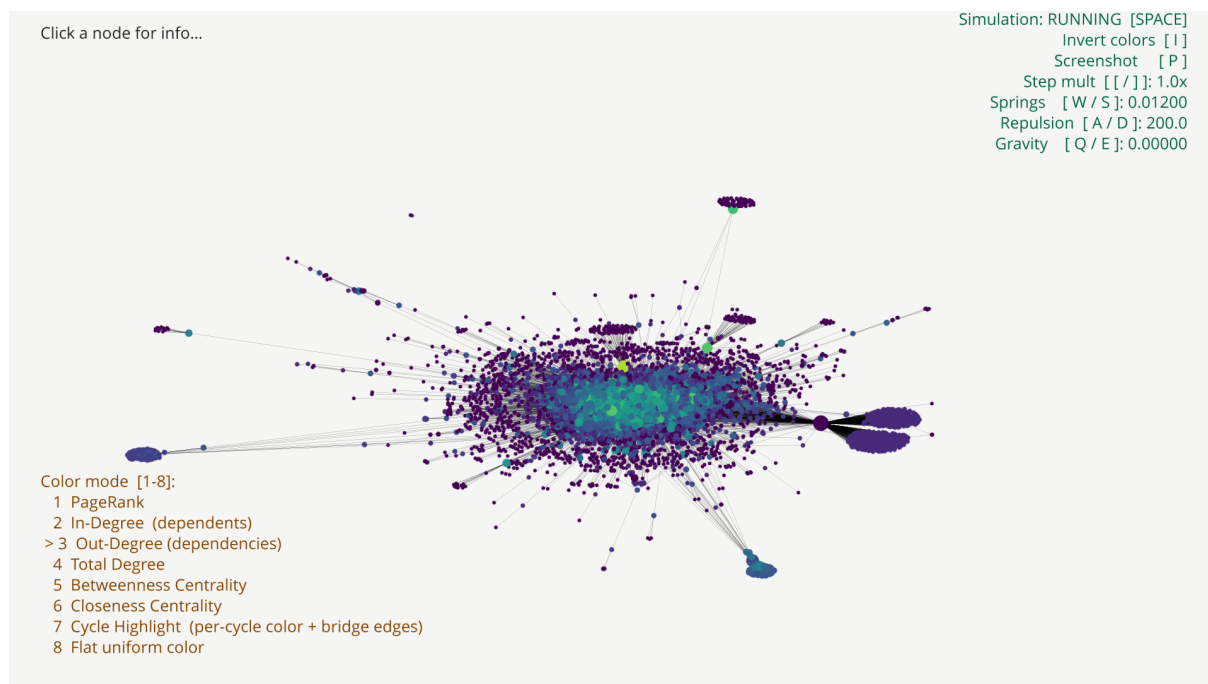


Figura 3.2: Representación de la red tras ejecutar el motor de físicas sobre la red hasta llegar al equilibrio.

3.4. Métricas y modos de color

Para agilizar la visualización de los datos en función de diferentes métricas o criterios se precálculan diferentes métricas y se almacenan normalizadas para representarse en diferentes escalas de color. Para alternar entre cada una de las visualizaciones se utilizan los números del 1 al 8. Los modos de color son los siguientes:

1. **PageRank**: con paleta *plasma*.
2. **Grado de entrada**: con paleta *inferno*.
3. **Grado de salida**: con paleta *viridis*.
4. **Grado total**: con paleta *magma*.
5. **Centralidad de intermediación** se utiliza una paleta con un degradado de negro a blanco pasando por rojo y amarillo.
6. **Centralidad de cercanía** se utiliza una paleta con un degradado de azul oscuro a cian.
7. **Destacado de ciclos**: se utiliza el método de la sección 3.5.
8. **Comunidades (Louvain)**: cada nodo se colorea según la comunidad a la que pertenece.

Las métricas de grado, PageRank, intermediación y cercanía se normalizan en escala logarítmica ($\log(1 + x)$).

3.5. Modo de visualización de ciclos

El modo 7 está dedicado al análisis de las componentes fuertemente conexas (SCC) del grafo, es decir, los conjuntos de paquetes que forman dependencias circulares entre sí.

Al activar este modo, se calculan todas las SCC con más de un nodo. A cada ciclo se le asigna un tono único extraído de una paleta generada distribuyendo uniformemente los matices del espacio HSV:

$$H_k = \frac{k}{N_{\text{ciclos}}}, \quad k = 0, 1, \dots, N_{\text{ciclos}} - 1 \quad (3.2)$$

Los nodos que no pertenecen a ningún ciclo se muestran en gris oscuro. Por otro lado, para añadir claridad se reestructura la coloración de las aristas:

- **Aristas intraciclo:** conexiones entre nodos del mismo ciclo. Se colorean con el tono de su ciclo, con transparencia parcial, de forma que las aristas interpolan visualmente entre los colores de los nodos que unen.
- **Aristas interciclo:** conexiones que van de un ciclo a otro ciclo distinto. Se muestran en dorado semitransparente.
- **Aristas al resto:** las conexiones que involucran al menos un nodo fuera de cualquier ciclo se ocultan.

3.6. Inversión de colores

La tecla I permite invertir la paleta de colores de toda la visualización. Se invierten los canales RGB de los nodos, el fondo pasa de negro a blanco y los colores del texto del se adaptan para mantener la legibilidad. Las aristas interciclo cambian del dorado al azul-violeta. Esta función resulta útil para capturas de pantalla.

3.7. Interacción

3.7.1. Controles de teclado

La totalidad de los parámetros de la simulación y la visualización es ajustable sin interrumpir la ejecución. La Tabla 3.1 resume los atajos disponibles.

Tabla 3.1: Atajos de teclado de la herramienta de visualización.

Tecla	Acción
ESPACIO	Activar / pausar la simulación de físicas
1–8	Cambiar el modo de coloración de nodos
I	Invertir la paleta de colores
W / S	Aumentar / reducir la rigidez de los muelles
A / D	Reducir / aumentar la fuerza de repulsión
Q / E	Aumentar / reducir la gravedad
[/]	Dividir / multiplicar por 10 el multiplicador

3.7.2. Inspección de nodos

Al hacer clic con el botón sobre un nodo se identifica el nodo más cercano al clic. Esto se realiza transformando las coordenadas de la pantalla al espacio del grafo mediante la inversa de la transformación de la cámara. Si la distancia al nodo más próximo es inferior a un umbral, se muestra en el texto superior izquierdo la siguiente información:

- Nombre del paquete.
- Grado de entrada y salida.
- Valor de la métrica activa para ese nodo.
- Identificador y tamaño del ciclo al que pertenece, o indicación de que el nodo no forma parte de ninguno.
- Identificador de la comunidad Louvain a la que pertenece el nodo.

Capítulo 4

Conclusiones

A continuación, se exponen las conclusiones del estudio:

- **Construcción de la red:** la red se construye por *web scrapping* a partir de la API de PyPI. Debido a la falta de preprocesamiento, algunas dependencias aparecen repetidas o con nombres erróneos.
- **Topología de la red:** la componente gigante abarca la totalidad de la red. Esta se organiza en grupos de dependencias que se interconectan entre sí a través de hubs. Existe un bucle que abarca aproximadamente la mitad de la red debido a que se han tenido en cuenta tanto las dependencias de producción como las de testeo y validación.
- **Distribución de grado:** el grado de entrada sigue una ley de potencias, conforme a la idea de que los paquetes populares cada vez se van a incluir más en los nuevos programas. Por su parte, el grado de salida no sigue una ley de potencias debido a que las dependencias de los nodos existentes tienden a mantenerse invariantes independientemente del crecimiento de la red.
- **Análisis de centralidad:** `typing-extensions`, `pytest` y `numpy` son los paquetes más importantes de acuerdo al PageRank, mientras que `pandas` lo es en términos de centralidad de intermediación. Se pone de manifiesto que las librerías de tipos y testeo de código son fundamentales para el correcto funcionamiento del resto de dependencias. Además, el manejo de datos tabulares resulta de gran interés en esta red.
- **Organización en comunidades:** el algoritmo de Louvain identifica comunidades relacionadas con entornos tecnológicos concretos. Entre otros: aprendizaje automático e inteligencia artificial, entorno AWS, desarrollo de APIs y servicios web...
- **Visualización:** el uso de `vispy` y la aceleración por GPU permite explorar redes de gran tamaño con un rendimiento superior a otros programas como Gephi. La herramienta desarrollada es útil para la identificación visual de anomalías, comunidades o nodos con una alta centralidad.

Bibliografía

- [1] A. Clauset, C. R. Shalizi, and M. E. J. Newman, “Power-law distributions in empirical data,” *SIAM Review*, vol. 51, no. 4, p. 661–703, Nov. 2009. [Online]. Available: <http://dx.doi.org/10.1137/070710111>
- [2] S. Martin, W. M. Brown, R. Klavans, and K. W. Boyack, “OpenOrd: An Open-Source Toolbox for Large Graph Layout,” in *Visualization and Data Analysis 2011*, P. C. Wong, J. Park, M. C. Hao, C. Chen, K. Börner, D. L. Kao, and J. C. Roberts, Eds., vol. 7868, International Society for Optics and Photonics. SPIE, 2011, p. 786806. [Online]. Available: <https://doi.org/10.1117/12.871402>